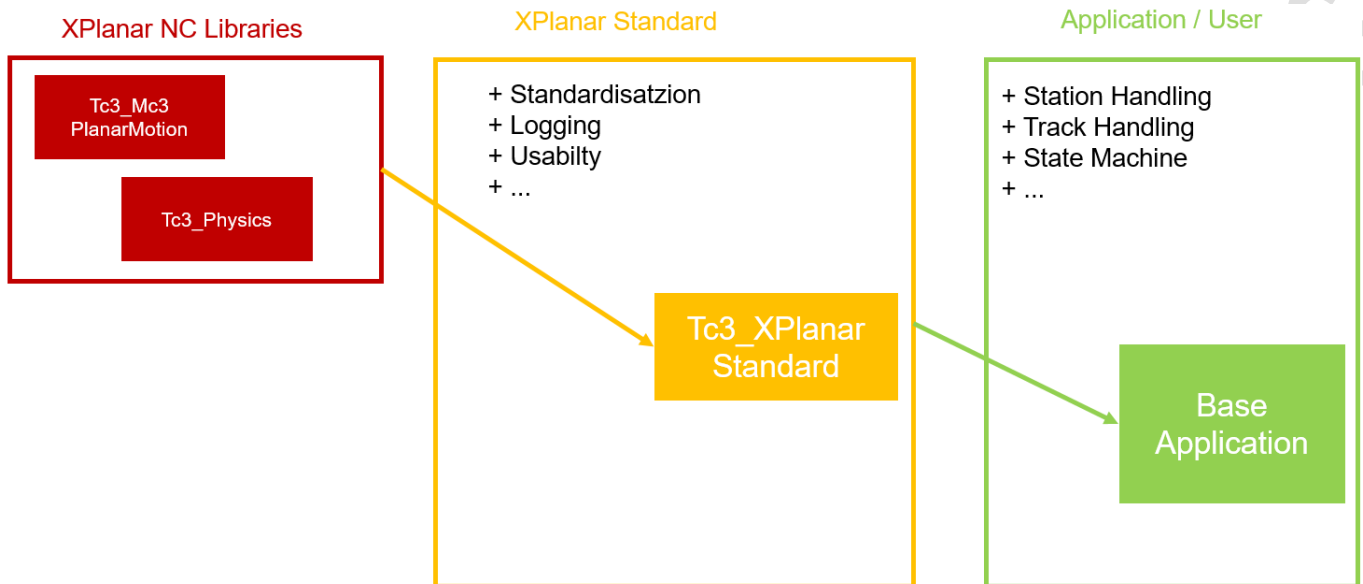


Tc3_XPlanarStandard

Introduction

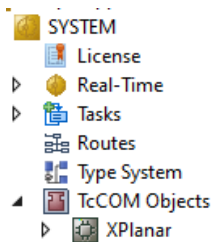
This library contains the essential objects to use in an application running an XPlanar system. Those objects serve as interfaces to the various elements involved in the application. The aim of the library is to encapsulate the necessary elements from the underlying NC libraries and extend their functionality spectrum somewhat.



XPlanar elements

XPlanar Processing Unit (aka XPlanar Driver)

The XPlanar driver is the brain behind the system. As a central intelligence, it controls the surface tiles (stator coils) which in turn make the movers levitate and move across the surface.



The XPlanar Processing Unit (XPU) can be found under the SYSTEM section in the TcCOM Objects list.

XPlanar Mover

Even though a mover is a passive element (permanent magnet) from a mechanical aspect, the software architecture considers a mover as an active NC axis. This allows a comprehensive organization and controlling, like classic motion applications. The mover is reflected in the library by the **FB_XPlanarMoverStandard** object.

XPlanar Track

A track is a one-dimensional movement path freely defined on the movement surface. This helps managing complex applications where multiple movers must move simultaneously. The track is reflected in the library by the **FB_TrackStandard** object.

Overview of the new functions

From V3.1.2204.0

- ▶ AdoptTrackOrientation command for FB_XPlanarMoverStandard
- ▶ P_ActVelocity property for FB_XPlanarMoverStandard
- ▶ Clear command for FB_TrackStandard

From V3.1.2203.1

- ▶ Requires TF5400 Advanced Motion Package V3.1.10.51 or superior
- ▶ Requires TF5890 XPlanar V3.2203.2.0 or superior
- ▶ C rotation feature for all mover types
- ▶ Logging improved
- ▶ Sorting improved for movers in FB_InitList (recovery sorting)

From V3.1.2012.37

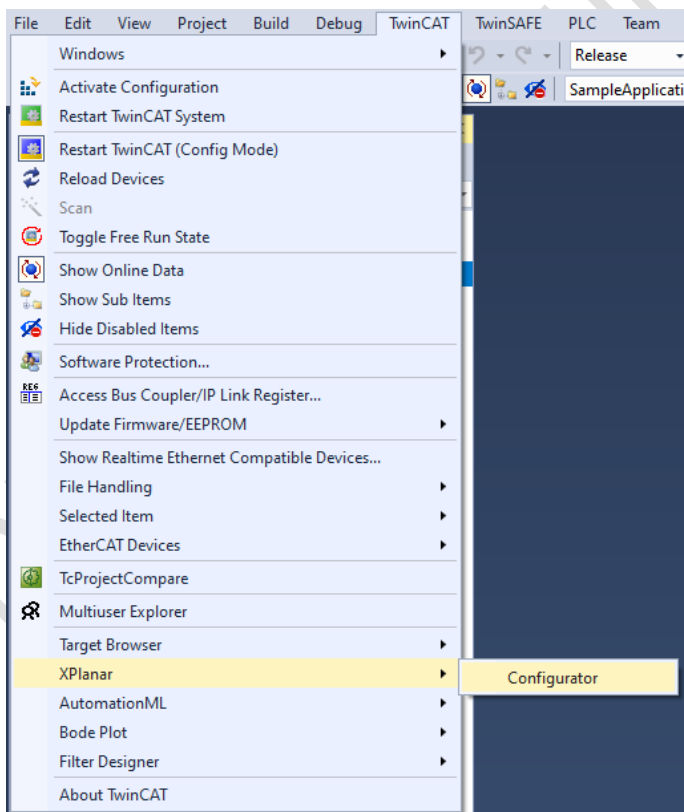
- ▶ BuildTrackExt method with extended checking

Quick Start

This chapter will guide you through the creation of a basic simulation project, to get familiar with the objects contained in the XPlanarStandard library.

Project definition

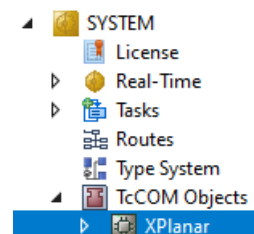
Start with a new project and define the XPlanar components using the XPlanar Configurator.



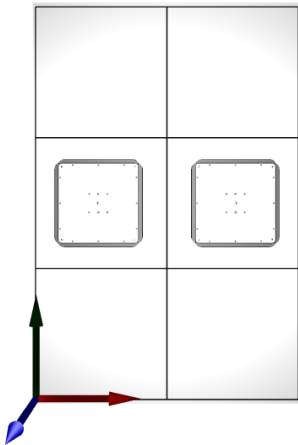
To do so, select **TwinCAT / XPlanar / Configurator** from the menu.

Configure the desired tiles and movers, then export the Configurator project.

The XPlanar TcCOM object should appear in the **SYSTEM** section under TcCOM Objects.



The example project will consider a system with 6 tiles (2 x 3 arrangement) and 2 x APM4330 movers.

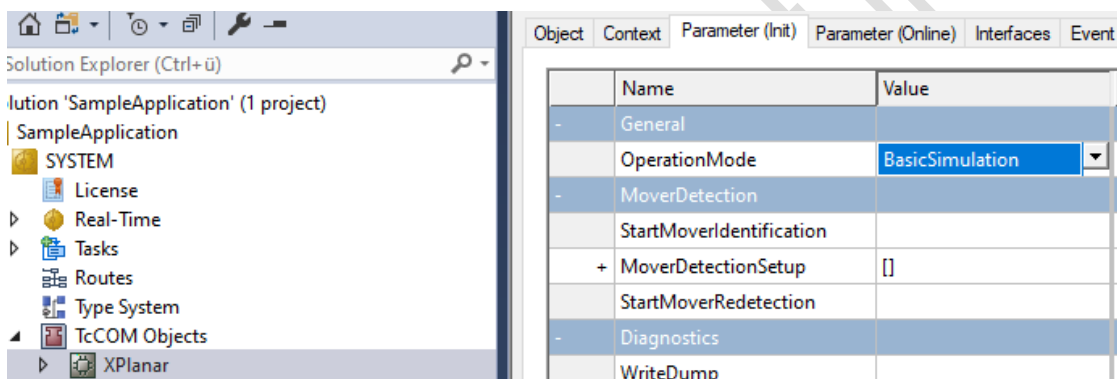


N.B. This layout corresponds to the APS9000 Starterkit.

Once the XPlanar configuration has been created, in our demo case we will set the XPlanar operation mode to BasicSimulation.

Select the **XPlanar** TcCOM object.

Change the **OperationMode** parameter to **BasicSimulation** in the **Parameter (Init)** tab.



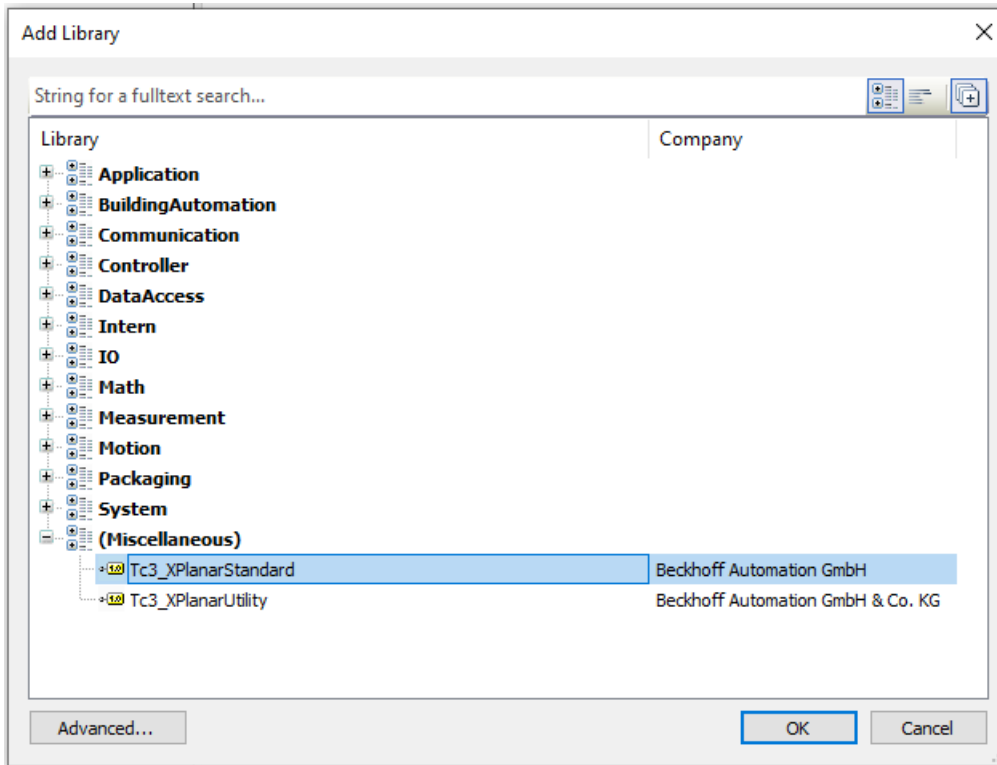
This mode allows the project to run on an engineering PC without connected hardware. The real-time specifications must not be met in this mode. All components can be grouped on a single core, but several cores are still possible (depending on the engineering PC used).

AdvancedSimulation mode is closer to the **Normal** operation mode by means of current and feedback calculations (active in this mode). The real-time specifications must be met in this mode, and the task layout shall be matching the real-world configuration (cycle time, load balance across cores, ...).

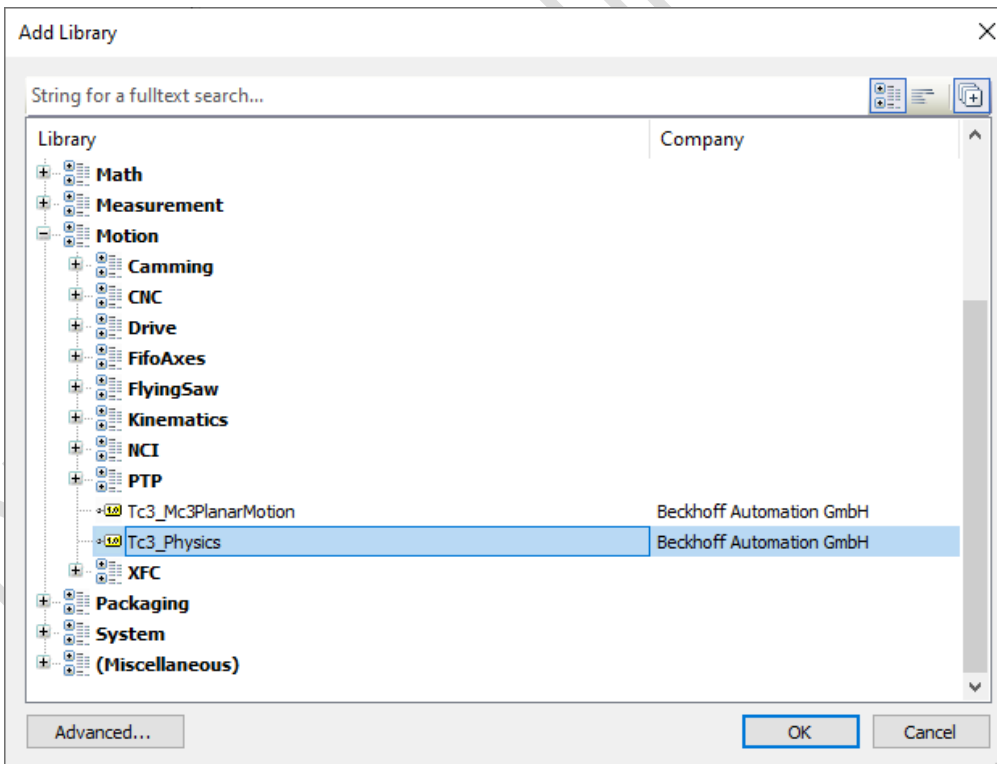
Normal operation mode requires the HW to be connected and communicating. This mode represents the situation on an actual system.

Create a new PLC project.

Add the **Tc3_XPlanarStandard** library to the project.

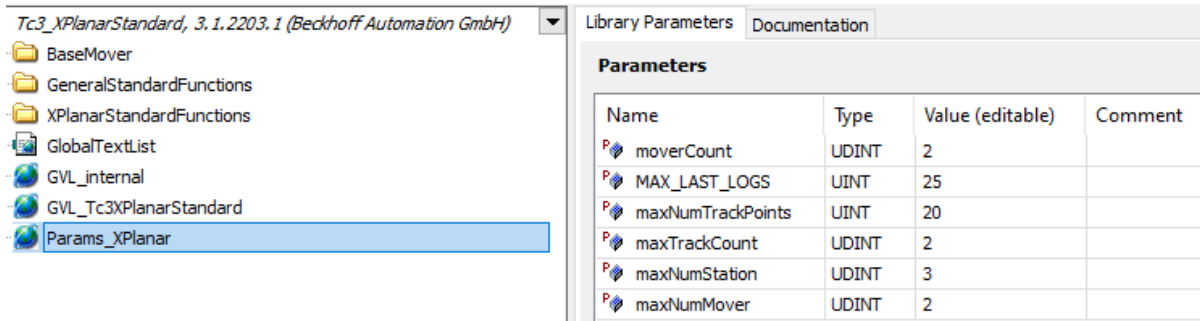


To get an easier access to additional required data types (declaration without prefix), also add the **Tc3_Physics** library to the project.

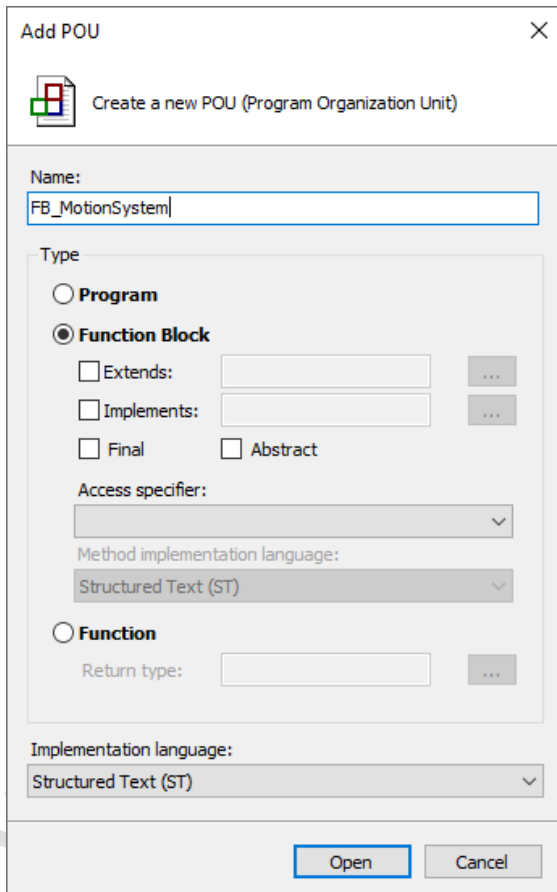


To define the correct quantities of objects used in the project, open the **Library Manager**, then select the **Tc3_XPlanarStandard** library.

Enter the desired values for movers, tracks, and stations (N.B. stations aren't used in this example) by double-clicking the fields, then replace their default values.



This will allow you to declare the arrays in your project with the proper size. In our example, we have added 2 movers with the XPlanar Configurator. Thus, the **maxNumMover** and **moverCount** parameters must be set accordingly. Later in this example we will also use 2 tracks, so **maxTrackCount** can be set too, if desired.



Add an object (function block) named FB_MotionSystem. This fb represents our XPlanar unit within a machine and will encapsulate all necessary elements to control it.

```
FUNCTION_BLOCK FB_MotionSystem
VAR
    fbXPlanarSystem      : FB_XPlanarSystem;
    fbXPlanarMover       : ARRAY[1..maxNumMover] OF FB_XPlanarMoverStandard;
END_VAR
```

Declare variables used to control the system.

- ▶ FB_XPlanarSystem controls the environment and can be seen as central management.
- ▶ FB_XPlanarMoverStandard represents a mover to be controlled (axis).

Syntax

```
FUNCTION_BLOCK FB_MotionSystem
VAR
    fbXPlanarSystem      := FB_XPlanarSystem;
    fbXPlanarMover       := ARRAY[1..maxNumMover] OF FB_XPlanarMoverStandard;
END_VAR
```

Hint

Please ensure that the object arrays are 1 based (not 0 based), as this indexing rule is applied throughout all XPlanar components.

For the objects to communicate and refresh their status, a cyclic update method must be called explicitly. This call is usually performed in MAIN, or any POU called by a task.

Add a method named **Cycle** with no return type to **FB_MotionSystem**. This method must be called in each PLC cycle whenever the fb is used.

The method will manage the cyclic status update of the **XPlanar** objects.

```
METHOD Cycle
VAR
    i : UDINT;
END_VAR

// Cyclic XPlanar objects update
fbXPlanarSystem.Cycle();

FOR i := 1 TO moverCount DO
    fbXPlanarMover[i].Cycle();
END_FOR
```

Note that in the declaration part we used the **maxNumMover** constant, but in the execution body we are using the **moverCount** parameter.

- **maxNumMover** is the designed quantity of movers for the application. It serves for creating the process image and size the arrays.
- **moverCount** is the actual quantity of movers controlled by the system. It can eventually be lower than the designed quantity if e.g., it is the result of a mover redetection operation (cf. **XPlanarUtility** lib documentation for more info), and movers have been removed from the tiles surface. Thus, it is recommended to use this variable in the execution body loops.

Syntax

```
METHOD Cycle
VAR
    i := UDINT;
END_VAR

// Cyclic XPlanar objects update
fbXPlanarSystem.Cycle();

FOR i := 1 TO moverCount DO
    fbXPlanarMover[i].Cycle();
END_FOR
```

```

PROGRAM MAIN
VAR
    fbXMotionUnit : FB_MotionSystem;
END_VAR

// Cyclic update
fbXMotionUnit.Cycle();

```

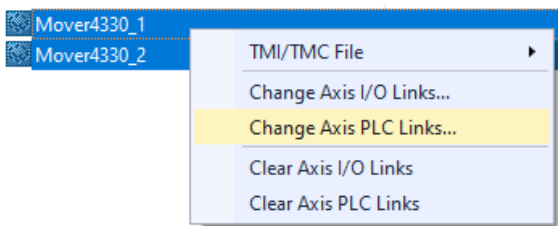
Declare an instance of the FB_MotionSystem in MAIN and call its Cycle method cyclically.

Build the project once.

After the successful build, the process image required for the XPlanar system to communicate should appear as shown below.

- ▲ PlcTask Inputs
 - ▶ MAIN.fbXMotionUnit.fbXPlanarSystem.environment.MCTOPLC_STD
 - ▶ MAIN.fbXMotionUnit.fbXPlanarSystem.group.MCTOPLC_STD
 - ▶ MAIN.fbXMotionUnit.fbXPlanarMover[1].stMoverRef.MCTOPLC
 - ▶ MAIN.fbXMotionUnit.fbXPlanarMover[2].stMoverRef.MCTOPLC
- ▲ PlcTask Outputs
 - ▶ MAIN.fbXMotionUnit.fbXPlanarMover[1].stMoverRef.PLCTOMC
 - ▶ MAIN.fbXMotionUnit.fbXPlanarMover[2].stMoverRef.PLCTOMC

Link all MCTOPLC as well as all PLCTOMC variables to their respective objects in the MOTION section.



The mover instances can be linked in the MOTION section at the **Axes** level. This links input and output structures simultaneously.

- ▲ Groups
 - ▲ XPlanar
 - ▶ McToPlc
 - ▶ STD
 - ▲ Environment
 - ▶ McToPlc
 - ▶ STD

The linking of C.A. group and environment is done as usual.

Order No	License	Instances
TC1200	TC3 PLC	cpu license
TF5000	TC3 NC PTP	cpu license
TF5890	TC3 XPlanar Technology	cpu license

Activate and run the project. If there are any missing licenses, TwinCAT will ask to generate new ones for 7 days (trial license). The required licenses to run a basic XPlanar application are:

- ▶ TC1250 TC3 PLC / NC PTP 10 (is equivalent to TC1200 TC3 PLC + TF5000 TC3 NC PTP)
- ▶ TF5890 TC3 XPlanar Technology

The state information as well as the actual values should update in the mover objects.

fbXMotionUnit	FB_MotionSystem				
fbXPlanarSystem	FB_XPlanarSystem				
fbXPlanarMover	ARRAY [1..maxNumMover] OF FB_XPlanarMoverStandard				
fbXPlanarMover[1]	FB_XPlanarMoverStandard				
stMoverRef	MC_PlanarMover				
PLCTOMC	CDT_PLCTOMC_PLANAR_MOVER				Mover data that is transferred from the Planar Mover to this function block.
MCTOPLC	CDT_MCTOPLC_PLANAR_MOVER				Mover data that is transferred from the Planar Mover to this function block.
STD	REFERENCE TO CDT_MCTOPLC_PLANAR_MOVER_STD				Mover standard data that is transferred from the Planar Mover to this function b
MoverOID	OTCID	16#05110010			Object id of the planar mover.
GroupOID	OTCID	16#00000000			Object id of the planar group the mover is in.
State	MC_PLANAR_STATE	Disabled			State of the planar mover, e.g. enabled.
CommandMode	MC_PLANAR_MOVER_COMMAND_MODE	FreeMovement			Command mode of the planar mover, e.g. onTrack.
Busy	MoverBusy				Busy state of the planar mover.
ErrorCode	HRESULT	16#00000000			Error code of the planar mover.
SET	REFERENCE TO CDT_MCTOPLC_PLANAR_MOVER_SET				Mover setpoint data that is transferred from the Planar Mover to this function b
ACT	REFERENCE TO CDT_MCTOPLC_PLANAR_MOVER_ACT				Mover actpoint data that is transferred from the Planar Mover to this function b
ActPos	MoverVector				Current position.
x	LREAL	119.99363824473777			X coordinate.
y	LREAL	360.00140591852312			Y coordinate.
z	LREAL	-0.01267314144736531			Z coordinate.
a	LREAL	0.0035828658200289291			A coordinate.
b	LREAL	0.0027642286314461723			B coordinate.
c	LREAL	0.0038556310254631368			C coordinate.
ActVelo	MoverVector				Current velocity.
DcTimeStamp	ULINT	0			Current time stamp.
PhysicalAreaID	UDINT	0			Current physical area id.
COORDMODE	REFERENCE TO CDT_MCTOPLC_PLANAR_MOVER_COORDMODE				Mover coordinate mode information that is transferred from the Planar Mover to
SETONTRACK	REFERENCE TO CDT_MCTOPLC_PLANAR_MOVER_TRACK				Mover busy information that is transferred from the Planar Mover to this functi
Error	BOOL	FALSE			Flag indicating a Planar Mover error.
ErrorId	UDINT	0			Error id indicating the Planar Mover error type.

Moving a mover

To bring a mover to levitate and move around over the tiles, a few preparation steps are necessary, as for any motion axis.

To keep things organized, a first sequence performing the administrative tasks at startup is to be programmed. This Startup sequence will be used when the application initializes. When done, the application can proceed, and it will no longer be used.

A second sequence handling the mover preparation is to be programmed next. This Starting sequence will take care of the mover preparation tasks, like enabling, ... These preparation tasks could be useful several times in the application's lifetime e.g., after an error stopped a mover, the user stopped the system, ...

A third sequence handling the actual movements execution is to be programmed. It becomes clear that this Execute sequence can only be performed when the Starting sequence completed, with the movers ready to operate.

Hint

The way XPlanar objects are addressed is based on sending a command (commands are found in form of methods in the objects), then polling the corresponding command feedback (feedback is found in form of properties in the objects) until it either succeeded (.Done flag) or failed (.Error flag). The cyclic update methods are necessary for the PLC objects to forward those commands to their respective external counterparts (NC objects, TcCOM objects), and refresh their internal status info.

If the cyclic update method of an object is not called in any PLC cycle, the expected command will not be sent to the XPlanar element and thus nothing will happen. Feedback info won't be updated either.

Add a method named Startup with a return type BOOL to FB_MotionSystem.

As this method is going to be a sequence, add a state machine variable **nStartupSequence** of type UDINT in the fb's declaration part. This variable will represent the current step in the sequence and can eventually be used to monitor the system's behavior.

When the method returns TRUE, it means the sequence completed successfully.

The sequence consists of passing the mover objects to the XPlanar system object, for it to control them in certain contexts.

```

METHOD Startup : BOOL
VAR
    i          : UDINT;
END_VAR

CASE nStartupSequence OF
    0: // Init
        nStartupSequence := nStartupSequence + 1;

    1: // Assign mover references to the system
        FOR i := 1 TO moverCount DO
            fbXPlanarSystem.SetMoverStandard(i, fbXPlanarMover[i]);
        END_FOR
        nStartupSequence := nStartupSequence + 29;

    30: // Done
        Startup := TRUE;
END_CASE

```

Syntax

```

METHOD Startup : BOOL
VAR
    i          : UDINT;
END_VAR

CASE nStartupSequence OF
    0: // Init
        nStartupSequence := nStartupSequence + 1;

    1: // Assign mover references to the system
        FOR i := 1 TO moverCount DO
            fbXPlanarSystem.SetMoverStandard(i, fbXPlanarMover[i]);

```

```

        END_FOR
        nStartupSequence := nStartupSequence + 29;

30:    // Done
        Startup := TRUE;
END_CASE

```

Add a method named Starting with a return type BOOL to FB_MotionSystem.

As this method is going to be a sequence, add a state machine variable **nStartingSequence** of type UDINT in the fb's declaration part. This variable will represent the current step in the sequence and can eventually be used to monitor the system's behavior.

When the method returns TRUE, it means the sequence completed successfully.

```

METHOD Starting : BOOL
VAR_INPUT
    bReset : BOOL; // Reset sequence
END_VAR

```

An input parameter bReset will allow the sequence to be reset by the application.

The sequence consists of enabling all movers present in the system. The Execute sequence is reset to ensure it will be executed from the beginning after Starting is complete.

```

METHOD Starting : BOOL
VAR_INPUT
    bReset : BOOL; // Reset sequence
END_VAR

IF bReset THEN
    Starting := FALSE;
    nStartingSequence := 0;
END_IF

CASE nStartingSequence OF
    0: // Init
        Execute(bReset:= TRUE);
        IF NOT bReset THEN
            nStartingSequence := nStartingSequence + 50;
        END_IF

    50: // Enable movers command
        IF fbXPlanarSystem.EnableMovers() THEN
            nStartingSequence := nStartingSequence + 1;
        END_IF

    51: // Enabling movers
        IF fbXPlanarSystem.P_EnableMovers.Done THEN
            nStartingSequence := nStartingSequence + 29;
        ELSIF fbXPlanarSystem.P_EnableTracks.Error THEN
            nStartingSequence := nStartingSequence + 4;
        END_IF

    80: // Done
        Starting := TRUE;
END_CASE

```

Syntax

```

METHOD Starting : BOOL
VAR_INPUT
    bReset: BOOL;      // Reset sequence
END_VAR

IF bReset THEN
    Starting := FALSE;
    nStartingSequence := 0;
END_IF

CASE nStartingSequence OF
    0:    // Init
        Execute(bReset:= TRUE);
        IF NOT bReset THEN
            nStartingSequence := nStartingSequence + 50;
        END_IF

    50:    // Enable movers command
        IF fbXPlanarSystem.EnableMovers() THEN
            nStartingSequence := nStartingSequence + 1;
        END_IF

    51:    // Enabling movers
        IF fbXPlanarSystem.P_EnableMovers.Done THEN

```

```

        nStartingSequence := nStartingSequence + 29;
    ELSIF fbXPlanarSystem.P_EnableTracks.Error THEN
        nStartingSequence := nStartingSequence + 4;
    END_IF

80:    // Done
        Starting := TRUE;
END_CASE

```

Add a method named Execute with a return type BOOL to FB_MotionSystem.

As this method is going to be a sequence, add a state machine variable **nExecutingSequence** of type UDINT in the fb's declaration part. This variable will represent the current step in the sequence and can eventually be used to monitor the system's behavior.

When the method returns TRUE, it means the sequence completed successfully. In our example, though, the sequence will never return TRUE as it could loop endlessly, until it gets aborted by an external condition (e.g. E-Stop on the system, user stops the machine, ...).

The sequence consists of moving both movers back and forth on the surface in Y direction.

To define the required target position for each mover, a variable of type **PositionXYC** (from the Tc3_Physics lib) is passed as argument to the MoveToPosition command.

```

METHOD Execute : BOOL
VAR_INPUT
    bReset          : BOOL; // Reset sequence
END_VAR
VAR
    i               : UDINT;
END_VAR
VAR_INST
    targetPos       : ARRAY[1..maxNumMover] OF PositionXYC;
    bMoversInPos    : BOOL;
    bMoverError     : BOOL;
END_VAR

IF bReset THEN
    Execute := FALSE;
    nExecutingSequence := 0;
END_IF

CASE nExecutingSequence OF
    0: // Init
        bMoversInPos := FALSE;
        bMoverError := FALSE;
        IF NOT bReset THEN
            IF fbXPlanarMover[1].P_CommandMode = MC_PLANAR_MOVER_COMMAND_MODE.FreeMovement THEN
                nExecutingSequence := nExecutingSequence + 10;
            END_IF
        END_IF

    10: // Move Mover 1 to X 120 mm Y 120 mm and Mover 2 to X 360 mm Y 600 mm with 500 mm/s
        targetPos[1].SetValuesXYC(120, 120, 0);
        targetPos[2].SetValuesXYC(360, 600, 0);
        FOR i := 1 TO moverCount DO
            fbXPlanarMover[i].MoveToPosition('Move to XY', targetPos[i], 500, 1250, 1250, 12500);
        END_FOR
        nExecutingSequence := nExecutingSequence + 1;

    11: // Movers moving
        bMoversInPos := TRUE;
        bMoverError := FALSE;
        FOR i := 1 TO moverCount DO
            bMoversInPos := bMoversInPos AND fbXPlanarMover[i].P_MoveToPosition.Done;
            bMoverError := bMoverError OR fbXPlanarMover[i].P_MoveToPosition.Error;
        END_FOR
        IF bMoversInPos THEN
            nExecutingSequence := nExecutingSequence + 9;
        ELSIF bMoverError THEN
            nExecutingSequence := nExecutingSequence + 4;
        END_IF

```

```

20: // Move Mover 1 to X 120 mm Y 600 mm and Mover 2 to X 360 mm Y 120 mm with 500 mm/s
targetPos[1].SetValuesXYC(120, 600, 0);
targetPos[2].SetValuesXYC(360, 120, 0);
FOR i := 1 TO moverCount DO
    fbXPlanarMover[i].MoveToPosition('Move to XY', targetPos[i], 500, 1250, 1250, 12500);
END_FOR
nExecutingSequence := nExecutingSequence + 1;

21: // Movers moving
bMoversInPos := TRUE;
bMoverError := FALSE;
FOR i := 1 TO moverCount DO
    bMoversInPos := bMoversInPos AND fbXPlanarMover[i].P_MoveToPosition.Done;
    bMoverError := bMoverError OR fbXPlanarMover[i].P_MoveToPosition.Error;
END_FOR
IF bMoversInPos THEN
    nExecutingSequence := nExecutingSequence + 9;
ELSIF bMoverError THEN
    nExecutingSequence := nExecutingSequence + 4;
END_IF

30: // Done
Execute := FALSE; // Operator must abort manually (via Stop or E-Stop)
nExecutingSequence := nExecutingSequence - 20; // Endless loop
END_CASE

```

Syntax

```

METHOD Execute : BOOL
VAR_INPUT
    bReset      : BOOL; // Reset sequence
END_VAR
VAR
    i           : UDINT;
END_VAR
VAR_INST
    targetPos   : ARRAY[1..maxNumMover] OF PositionXYC;
    bMoversInPos : BOOL;
    bMoverError  : BOOL;
END_VAR

IF bReset THEN
    Execute := FALSE;
    nExecutingSequence := 0;
END_IF

CASE nExecutingSequence OF
    0: // Init
        bMoversInPos := FALSE;
        bMoverError := FALSE;
        IF NOT bReset THEN
            IF fbXPlanarMover[1].P_CommandMode =
MC_PLANAR_MOVER_COMMAND_MODE.FreeMovement THEN
                nExecutingSequence := nExecutingSequence + 10;
            END_IF
        END_IF

        10: // Move Mover 1 to X 120 mm Y 120 mm and Mover 2 to X 360 mm Y 600 mm with
500 mm/s
            targetPos[1].SetValuesXYC(120, 120, 0);

```

```

        targetPos[2].SetValuesXYC(360, 600, 0);
        FOR i := 1 TO moverCount DO
            fbXPlanarMover[i].MoveToPosition('Move to XY', targetPos[i], 500,
1250, 1250, 12500);
        END_FOR
        nExecutingSequence := nExecutingSequence + 1;

11:    // Movers moving
        bMoversInPos := TRUE;
        bMoverError := FALSE;
        FOR i := 1 TO moverCount DO
            bMoversInPos := bMoversInPos AND
fbXPlanarMover[i].P_MoveToPosition.Done;
            bMoverError := bMoverError OR
fbXPlanarMover[i].P_MoveToPosition.Error;
        END_FOR
        IF bMoversInPos THEN
            nExecutingSequence := nExecutingSequence + 9;
        ELSIF bMoverError THEN
            nExecutingSequence := nExecutingSequence + 4;
        END_IF

20:    // Move Mover 1 to X 120 mm Y 600 mm and Mover 2 to X 360 mm Y 120 mm with
500 mm/s
        targetPos[1].SetValuesXYC(120, 600, 0);
        targetPos[2].SetValuesXYC(360, 120, 0);
        FOR i := 1 TO moverCount DO
            fbXPlanarMover[i].MoveToPosition('Move to XY', targetPos[i], 500,
1250, 1250, 12500);
        END_FOR
        nExecutingSequence := nExecutingSequence + 1;

21:    // Movers moving
        bMoversInPos := TRUE;
        bMoverError := FALSE;
        FOR i := 1 TO moverCount DO
            bMoversInPos := bMoversInPos AND
fbXPlanarMover[i].P_MoveToPosition.Done;
            bMoverError := bMoverError OR
fbXPlanarMover[i].P_MoveToPosition.Error;
        END_FOR
        IF bMoversInPos THEN
            nExecutingSequence := nExecutingSequence + 9;
        ELSIF bMoverError THEN
            nExecutingSequence := nExecutingSequence + 4;
        END_IF

30:    // Done
        Execute := FALSE; // Operator must abort manually (via Stop or E-Stop)
        nExecutingSequence := nExecutingSequence - 20; // Endless loop
END_CASE

```

When state 30 is reached, the sequence jumps back to state 10 to loop endlessly.

Hint

Mover positions always refer to the mover's center point. Depending on the mover's size (mover model, tooling, payload) a proper distancing must be considered when commanding movements in the application, incl. distance from the surface boundaries and gap between movers (cf. Collision Avoidance).

```

PROGRAM MAIN
VAR
    fbXMotionUnit    : FB_MotionSystem;
    nState           : UDINT;
END_VAR

CASE nState OF
    0: // Init
        nState := nState + 1;

    1: // Startup sequence
        IF fbXMotionUnit.Startup() THEN
            nState := nState + 29;
        END_IF

    30: // Preparing sequence
        IF fbXMotionUnit.Starting(bReset:= FALSE) THEN
            nState := nState + 10;
        END_IF

    40: // Executing sequence
        fbXMotionUnit.Execute(bReset:= FALSE);
END_CASE

// Cyclic update
fbXMotionUnit.Cycle();

```

To run the application and call the sequences in the correct order, a state machine is to be programmed in MAIN.

Since the Execute sequence is an endless loop in our case, there is no further step after reaching the Execute state. A manual aborting must be considered (not implemented in this example) to leave that state.

Syntax

```

PROGRAM MAIN
VAR
    fbXMotionUnit    : FB_MotionSystem;
    nState           : UDINT;
END_VAR

CASE nState OF
    0: // Init
        nState := nState + 1;

    1: // Startup sequence
        IF fbXMotionUnit.Startup() THEN
            nState := nState + 29;
        END_IF

    30: // Preparing sequence
        IF fbXMotionUnit.Starting(bReset:= FALSE) THEN
            nState := nState + 10;
        END_IF

    40: // Executing sequence
        fbXMotionUnit.Execute(bReset:= FALSE);
END_CASE

// Cyclic update
fbXMotionUnit.Cycle();

```

When running the application, the current sequence steps can be monitored via the implemented state machines.

fbXMotionUnit	FB_MotionSystem	
fbXPlanarSystem	FB_XPlanarSystem	
fbXPlanarMover	ARRAY [1..maxNumMover] OF FB_XPlanarMoverStandard	
fbTrackStandard	ARRAY [1..maxTrackCount] OF FB_TrackStandard	
nStartupSequence	UDINT	30
nStartingSequence	UDINT	80
nRecoverySequence	UDINT	0
nExecutingSequence	UDINT	11
nState	UDINT	40

It can be observed, once the Execute state has been reached by the application, that the nExecutingSequence is endlessly looping from steps 10 to 30, indicating the back-and-forth movements of the movers.

```

1 CASE nState 40 OF
2   0: // Init
3     nState 40 := nState 40 + 1;
4
5   1: // Startup sequence
6     IF fbXMotionUnit.Startup() THEN
7       nState 40 := nState 40 + 29;
8     END_IF
9
10  30: // Preparing sequence
11    IF fbXMotionUnit.Starting(bReset:= FALSE) THEN
12      nState 40 := nState 40 + 10;
13    END_IF
14
15  40: // Executing sequence
16    fbXMotionUnit.Execute(bReset:= FALSE);
17 END_CASE
18
19 // Cyclic update
20 fbXMotionUnit.Cycle();
21 RETURN

```

By using the XPlanar Configurator in Live-View mode, the movers can be monitored as well.

Logger

To help the programmer check for proper operations, a logger is implemented in each object. As it could be observed in the example, each command contains a log entry message as first parameter.

The logger can be seen online in the object (e.g., the mover), to verify what commands have been performed.

aLog	ARRAY [0..MAX_LAST_LOGS] OF ST_MoverCommandLog	
aLog[0]	ST_MoverCommandLog	
aLog[1]	ST_MoverCommandLog	
nNo	INT	1
sTimeStamp	STRING	'2022-04-05-10:48:51.556025900'
sTimeStampDate	STRING	'2022-04-05'
sTimeStampTime	STRING	'10:48:51.556'
sStation	STRING	'Move to XY'
sCommandType	STRING	'MoveToPosition'
sText	STRING(255)	'TargetPos: X:120.0;Y:600.0;C:0.0;Velo:500.00;Acc:1250;Dec:1250;Jerk:12500'
aLog[2]	ST_MoverCommandLog	

The quantity of logging entries can be set by modifying the parameter **MAX_LAST_LOGS** in the library's parameters. The index 1 is always the most recent entry in the logging list.

Collision Avoidance

In the previous example, the movers were commanded without any kind of collision avoidance, so that they were able to collide, or fall from the tiles surface if commanded to.

To take advantage of the central intelligence of the XPlanar system and avoid colliding with another object or leaving the movement area whenever a move command is sent, all involved elements can be added together to a so-called Collision Avoidance Group (referred to as "Group"), which will verify the proper conditions to successfully execute a command prior to execute it, and eventually report an error if conditions for a proper execution aren't met. This makes each object aware of its neighborhood, so to say.

The elements that can be added to the Group for Collision Avoidance monitoring are:

- ▶ Movers
- ▶ Environment (represents the stator surface boundaries)
- ▶ Tracks (will be explained later in this chapter)

Hint

Elements (except environment boundaries, which is a static object) must be in state **Enabled** before they can be added to the Collision Avoidance group. Disabled elements are thus not considered in the monitoring conditions. Whenever a mover is commanded by an external setpoint generator it attempts to follow that setpoint regardless of any Collision Avoidance monitoring influence. The mover thus runs without Collision Avoidance limitations for as long as the external setpoint generation mode is active.

Now, to perform the same movement as before, but with Collision Avoidance monitoring active, let's re-write our previous sequences and add our movers as well as the surface boundaries to the C.A. Group before any movement. This way, collision between movers and/or movements outside of the surface boundaries are going to be prevented.

The Startup sequence remains unchanged. The C.A. group and the stator surface boundaries are integral part of the FB_XPlanarSystem object.

```

METHOD Starting : BOOL
VAR_INPUT
    bReset : BOOL; // Reset sequence
END_VAR

IF bReset THEN
    Starting := FALSE;
    nStartingSequence := 0;
END_IF

CASE nStartingSequence OF
    0: // Init
        Execute(bReset:= TRUE);
        MoverRecovery(bReset:= TRUE);
        IF NOT bReset THEN
            nStartingSequence := nStartingSequence + 10;
        END_IF

    10: // Enable C.A. group command
        IF fbXPlanarSystem.EnableGroup() THEN
            nStartingSequence := nStartingSequence + 1;
        END_IF

    11: // Enabling group
        IF fbXPlanarSystem.P_EnableGroup.Done THEN
            nStartingSequence := nStartingSequence + 9;
        ELSIF fbXPlanarSystem.P_EnableGroup.Error THEN
            nStartingSequence := nStartingSequence + 4;
        END_IF

    20: // Add environment (surface boundaries) to group command
        IF fbXPlanarSystem.AddEnvironmentToGroup() THEN
            nStartingSequence := nStartingSequence + 1;
        END_IF

    21: // Adding environment to group
        IF fbXPlanarSystem.P_AddEnviromentToGroup.Done THEN
            nStartingSequence := nStartingSequence + 29;
        ELSIF fbXPlanarSystem.P_AddEnviromentToGroup.Error THEN
            nStartingSequence := nStartingSequence + 4;
        END_IF

    51: // Enabling movers
        IF fbXPlanarSystem.P_EnableMovers.Done THEN
            nStartingSequence := nStartingSequence + 9;
        ELSIF fbXPlanarSystem.P_EnableTracks.Error THEN
            nStartingSequence := nStartingSequence + 4;
        END_IF

    60: // Add movers to group command
        IF fbXPlanarSystem.AddMoversToGroup() THEN
            nStartingSequence := nStartingSequence + 1;
        END_IF

    61: // Adding movers to group
        IF fbXPlanarSystem.P_AddMoversToGroup.Done THEN
            nStartingSequence := nStartingSequence + 19;
        ELSIF fbXPlanarSystem.P_AddMoversToGroup.Error THEN
            nStartingSequence := nStartingSequence + 4;
        END_IF

    80: // Done
        Starting := TRUE;
END_CASE

```

Modify the Starting sequence as shown to perform the additional preparation tasks.

- ▶ Collision Avoidance Group gets enabled
- ▶ The stator surface boundaries (Environment) object gets included in the group
- ▶ Movers get enabled (already implemented)
- ▶ Movers get included in the group

The order of operations in the Starting sequence is important and must be observed.

Syntax

```

METHOD Starting : BOOL
VAR_INPUT

```

```

        bReset: BOOL;          // Reset sequence
END_VAR
IF bReset THEN
    Starting := FALSE;
    nStartingSequence := 0;
END_IF

CASE nStartingSequence OF
    0:    // Init
        Execute(bReset:= TRUE);
        IF NOT bReset THEN
            nStartingSequence := nStartingSequence + 10;
        END_IF

    10:   // Enable C.A. group command
        IF fbXPlanarSystem.EnableGroup() THEN
            nStartingSequence := nStartingSequence + 1;
        END_IF

    11:   // Enabling group
        IF fbXPlanarSystem.P_EnableGroup.Done THEN
            nStartingSequence := nStartingSequence + 9;
        ELSIF fbXPlanarSystem.P_EnableGroup.Error THEN
            nStartingSequence := nStartingSequence + 4;
        END_IF

    20:   // Add environment (surface boundaries) to group command
        IF fbXPlanarSystem.AddEnvironmentToGroup() THEN
            nStartingSequence := nStartingSequence + 1;
        END_IF

    21:   // Adding environment to group
        IF fbXPlanarSystem.P_AddEnviromentToGroup.Done THEN
            nStartingSequence := nStartingSequence + 29;
        ELSIF fbXPlanarSystem.P_AddEnviromentToGroup.Error THEN
            nStartingSequence := nStartingSequence + 4;
        END_IF

    50:   // Enable movers command
        IF fbXPlanarSystem.EnableMovers() THEN
            nStartingSequence := nStartingSequence + 1;
        END_IF

    51:   // Enabling movers
        IF fbXPlanarSystem.P_EnableMovers.Done THEN
            nStartingSequence := nStartingSequence + 9;
        ELSIF fbXPlanarSystem.P_EnableTracks.Error THEN
            nStartingSequence := nStartingSequence + 4;
        END_IF

    60:   // Add movers to group command
        IF fbXPlanarSystem.AddMoversToGroup() THEN

```

```

        nStartingSequence := nStartingSequence + 1;
    END_IF

61:    // Adding movers to group
    IF fbXPlanarSystem.P_AddMoversToGroup.Done THEN
        nStartingSequence := nStartingSequence + 19;
    ELSIF fbXPlanarSystem.P_AddMoversToGroup.Error THEN
        nStartingSequence := nStartingSequence + 4;
    END_IF

80:    // Done
    Starting := TRUE;

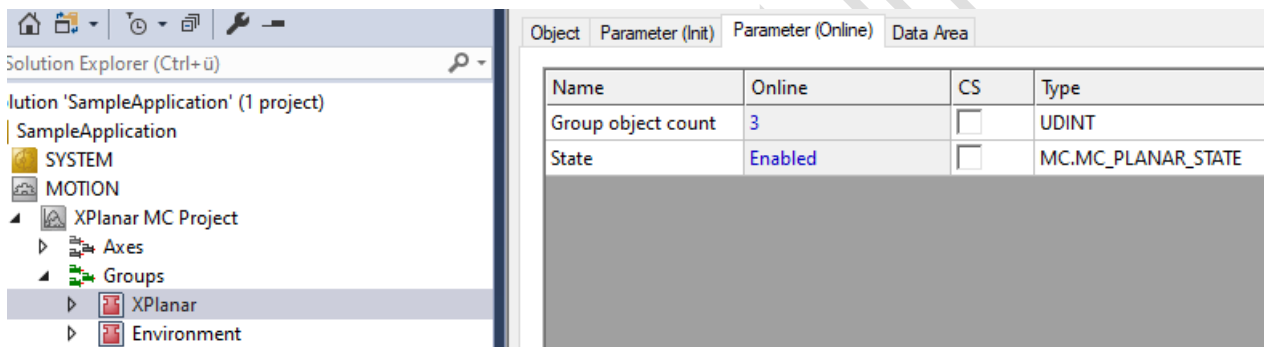
END_CASE

```

The Execute sequence remains unchanged, as we keep the same commanded movements as before.

Test the modified application by reactivating the configuration and reloading the project in the PLC. The sequence should still cycle endlessly as before.

By verifying the Collision Avoidance group's **GroupObjectCount** property, we can see the quantity of objects taken into consideration for the avoidance mechanism.

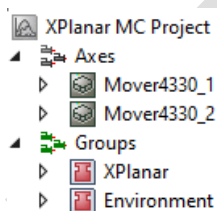


The screenshot shows the Beckhoff XPlanar software interface. On the left, the 'Solution Explorer (Ctrl+u)' displays the project structure: 'SampleApplication' (1 project) containing 'SYSTEM' and 'MOTION'. Under 'MOTION', there is an 'XPlanar MC Project' which includes 'Axes', 'Groups', 'XPlanar', and 'Environment'. The 'Groups' folder is expanded, showing 'XPlanar' and 'Environment'. On the right, the 'Properties' window is open, showing the 'Parameter (Online)' tab. It contains a table with the following data:

Name	Online	CS	Type
Group object count	3	☐	UDINT
State	Enabled	☐	MC_MC_PLANAR_STATE

Here 3 objects are part of the group:

- ▶ 2 x mover object
- ▶ 1 x environment (representing the surface boundaries) object



The screenshot shows the 'Solution Explorer' with the 'XPlanar MC Project' expanded. It displays the following structure:

- XPlanar MC Project
 - Axes
 - Mover4330_1
 - Mover4330_2
 - Groups
 - XPlanar
 - Environment

The **XPlanar** Collision Avoidance group object as well as the **Environment** object can both be found in the MOTION section under **Groups**.

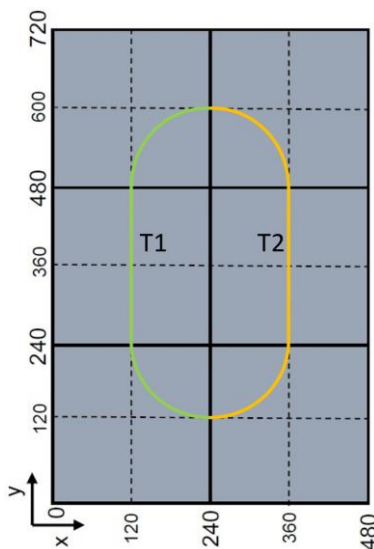
Working with tracks

To ease the mover commanding process when handling several movers simultaneously, the concept of one-dimensional tracks, freely designable across the surface, has been implemented. These tracks allow movers to move along a defined path, similarly to e.g., an XTS system or any chain or belt conveying system.

A track is nothing more than a user defined one-dimensional (linear or curved) path with a start point and an end point, anywhere on the stator surface, that movers can follow. An analogy to trains and railroad tracks can be made to understand that concept and guide the application design.

While a single track can be helpful in some applications (additionally the track can, if desired, be defined as a closed loop, allowing modulo movement patterns), building a track network by linking a track's start point to its predecessor's end point will be even more helpful, opening new possibilities. A track network can provide a foundation for routing movers towards different targets, allow for more complex geometries, ...

To handle complex mover routing requirements in an application, another analogy to the train management is made by means of implementing stations, each receiving / sending away movers to their subsequent destination.



Here an application example using a track network of 2 tracks (T1, T2) laid out at defined coordinates on the stator surface is shown.

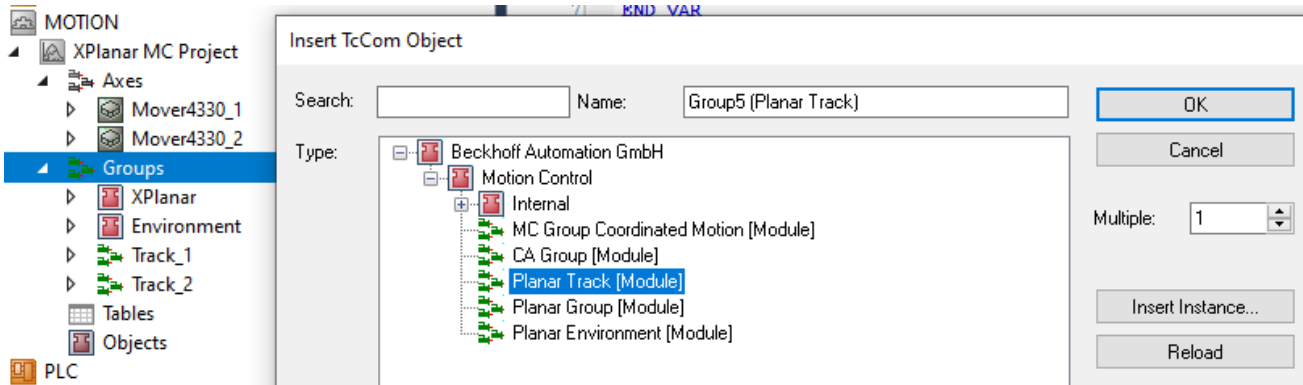
Movers are moved around the tracks simultaneously.

Collision Avoidance can be utilized even further in this situation by queueing up movers whenever necessary. Each mover will keep a safe distance (= gap) to its predecessor along the track.

Declaring tracks

In an XPlanar system tracks appear in form of NC group objects. As the organization implies it, they will be part of the Collision Avoidance mechanism like the Environment (surface boundaries) object is.

Under Groups in the MOTION section, add as many track objects as required and name them accordingly. In our example we defined the parameter **maxTrackCount** to 2, so 2 tracks must be added in this section.



```
FUNCTION_BLOCK FB_MotionSystem
VAR
    fbXPlanarSystem      : FB_XPlanarSystem;
    fbXPlanarMover       : ARRAY[1..maxNumMover] OF FB_XPlanarMoverStandard;
    fbTrackStandard      : ARRAY[1..maxTrackCount] OF FB_TrackStandard;

    nStartupSequence     : UDINT;
    nStartingSequence    : UDINT;
```

Declare an array of objects of type **FB_TrackStandard** in the motion system fb.

Syntax

```
FUNCTION_BLOCK FB_MotionSystem
VAR
    fbXPlanarSystem      : FB_XPlanarSystem;
    fbXPlanarMover       : ARRAY[1..maxNumMover] OF FB_XPlanarMoverStandard;
    fbTrackStandard      : ARRAY[1..maxTrackCount] OF FB_TrackStandard;

    nStartupSequence     : UDINT;
    nStartingSequence    : UDINT;
```

```
METHOD Cycle
VAR
    i      : UDINT;
END_VAR

// Cyclic XPlanar objects update
fbXPlanarSystem.Cycle();

FOR i := 1 TO moverCount DO
    fbXPlanarMover[i].Cycle();
END_FOR

FOR i := 1 TO maxTrackCount DO
    fbTrackStandard[i].Cycle();
END_FOR
```

Add the tracks cyclic update calls in the Cycle method.

Syntax

```
// Cyclic XPlanar objects update
fbXPlanarSystem.Cycle();

FOR i := 1 TO moverCount DO
    fbXPlanarMover[i].Cycle();
END_FOR

FOR i := 1 TO maxTrackCount DO
    fbTrackStandard[i].Cycle();
END_FOR
```

As for the movers, to allow the XPlanar system object to interact with the declared tracks, they must be passed to the system object at application startup.

```
CASE nStartupSequence OF
  0: // Init
    nStartupSequence := nStartupSequence + 1;

  1: // Assign mover references to the system
    FOR i := 1 TO moverCount DO
        fbXPlanarSystem.SetMoverStandard(i, fbXPlanarMover[i]);
    END_FOR
    nStartupSequence := nStartupSequence + 1;

  2: // Assign track references to the system
    FOR i := 1 TO maxTrackCount DO
        fbXPlanarSystem.SetTrackStandard(i, fbTrackStandard[i]);
    END_FOR
    nStartupSequence := nStartupSequence + 28;

  30: // Done
    Startup := TRUE;
END_CASE
```

Modify the Startup sequence to assign the track references to the system, as done with the movers.

Syntax

```
CASE nStartupSequence OF
  0: // Init
    nStartupSequence := nStartupSequence + 1;

  1: // Assign mover references to the system
    FOR i := 1 TO moverCount DO
        fbXPlanarSystem.SetMoverStandard(i, fbXPlanarMover[i]);
    END_FOR
    nStartupSequence := nStartupSequence + 1;

  2: // Assign track references to the system
    FOR i := 1 TO maxTrackCount DO
        fbXPlanarSystem.SetTrackStandard(i, fbTrackStandard[i]);
    END_FOR
    nStartupSequence := nStartupSequence + 28;

  30: // Done
    Startup := TRUE;
END_CASE
```

Rebuild the project, link the variables to the newly created objects in the MOTION section, then reactivate the configuration. The track objects state should now refresh.

fbTrackStandard[1]	FB_TrackStandard	
Busy	BOOL	FALSE
stTrackRef	MC_PlanarTrack	
MCTOPLC_STD	CDT_MCTOPLC_PLANAR_TRACK	
TrackOID	OTCID	16#05120030
GroupOID	OTCID	16#00000000
State	MC_PLANAR_STATE	Disabled
OperationMode	MC_PLANAR_TRACK_OPERATION_MODE	Uninitialized
MoverCountOnTrack	UDINT	0
MovingMoverCount	UDINT	0
Error	BOOL	FALSE
ErrorId	UDINT	0

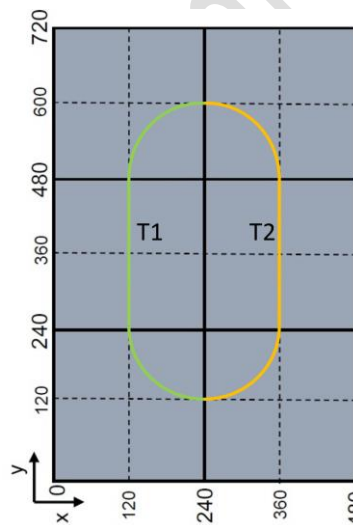
Before movers can make use of them, tracks must first be designed and built in the application. Designing and building the tracks occurs at application startup. Once the tracks are built, they remain existing for the application's lifetime.

Designing a track

To design a track, we first start with a sketch. Locate the track segments' coordinates X and Y. The number of segments depends on each track's complexity (shape) and relationships, in case of track networks.

Track T1		
Segment	X (mm)	Y (mm)
Point	240	120
Line Start	120	240
Line End	120	480
Point	240	600

Track T2		
Segment	X (mm)	Y (mm)
Line Start	360	480
Line End	360	240
StartFromTrack := 1		
EndAtTrack := 1		



Since track design occurs at application startup, the Startup method will evolve once again, to include the necessary steps to build the tracks.

```

METHOD Startup : BOOL
VAR
    i                : UDINT;
END_VAR
VAR_INST
    aTrackTable      : ARRAY[1..maxTrackCount] OF FB_TrackTable;
    nTrackIdx         : UDINT;

    nStartFromTrack  : UDINT;
    nEndAtTrack       : UDINT;
    bCloseLoop        : BOOL;
END_VAR

```

To contain the listed segments above, a table of coordinates (points) is established for each track.

To define the track tables, add an array of **FB_TrackTable** helper objects to the Startup method.

Syntax

```
METHOD Startup : BOOL
VAR
    i                : UDINT;
END_VAR
VAR_INST
    aTrackTable      : ARRAY[1..maxTrackCount] OF FB_TrackTable;
    nTrackIdx        : UDINT;

    nStartFromTrack   : UDINT;
    nEndAtTrack       : UDINT;
    bCloseLoop        : BOOL;
END_VAR
```

Hint

A track geometry must exist (track successfully built) before any subsequent track can use it as predecessor to form a track network. The order of track building is therefore important and must be observed.

It is highly recommended to build one track at a time, to avoid errors in track networks.

```

METHOD Startup : BOOL
VAR
    i                : UDINT;
END_VAR
VAR_INST
    aTrackTable      : ARRAY[1..maxTrackCount] OF FB_TrackTable;
    nTrackIdx         : UDINT;

    nStartFromTrack   : UDINT;
    nEndAtTrack       : UDINT;
    bCloseLoop        : BOOL;
END_VAR
=====
CASE nStartupSequence OF
    0: // Init
        nStartupSequence := nStartupSequence + 1;

    1: // Assign mover references to the system
        FOR i := 1 TO moverCount DO
            fbXPlanarSystem.SetMoverStandard(i, fbXPlanarMover[i]);
        END_FOR
        nStartupSequence := nStartupSequence + 1;

    2: // Assign track references to the system
        FOR i := 1 TO maxTrackCount DO
            fbXPlanarSystem.SetTrackStandard(i, fbTrackStandard[i]);
        END_FOR
        nStartupSequence := nStartupSequence + 8;

    10: // Build track geometries
        nTrackIdx := 0;
        nStartupSequence := nStartupSequence + 1;

    11: // Start with an empty table
        nTrackIdx := nTrackIdx + 1;
        IF nTrackIdx > maxTrackCount THEN
            // All tracks are built
            nStartupSequence := nStartupSequence + 19;
        ELSE
            aTrackTable[nTrackIdx].ClearTrackTable();
            nStartupSequence := nStartupSequence + 1;
        END_IF
END_CASE

```

```

12: // Fill track table with segments (qty of segments depends on each track)
CASE nTrackIdx OF
  1: // T1
    nStartFromTrack := 0;
    nEndAtTrack := 0;
    bCloseLoop := FALSE;

    aTrackTable[nTrackIdx].AddPoint(E_PointType.Point,      E_PointOption.none, 240, 120, 0);

    aTrackTable[nTrackIdx].AddPoint(E_PointType.Line_Start, E_PointOption.none, 120, 240, 0);
    aTrackTable[nTrackIdx].AddPoint(E_PointType.Line_End,   E_PointOption.none, 120, 480, 0);

    aTrackTable[nTrackIdx].AddPoint(E_PointType.Point,      E_PointOption.none, 240, 600, 0);

  2: // T2
    nStartFromTrack := 1;
    nEndAtTrack := 1;
    bCloseLoop := FALSE;

    aTrackTable[nTrackIdx].AddPoint(E_PointType.Line_Start, E_PointOption.none, 360, 480, 0);
    aTrackTable[nTrackIdx].AddPoint(E_PointType.Line_End,   E_PointOption.none, 360, 240, 0);
END_CASE
nStartupSequence := nStartupSequence + 1;

13: // Set additional design info
IF nStartFromTrack > 0 THEN
  aTrackTable[nTrackIdx].P_StartFromTrack := fbTrackStandard[nStartFromTrack];
END_IF
IF nEndAtTrack > 0 THEN
  aTrackTable[nTrackIdx].P_EndAtTrack := fbTrackStandard[nEndAtTrack];
END_IF
IF bCloseLoop AND nStartFromTrack = 0 AND nEndAtTrack = 0 THEN
  aTrackTable[nTrackIdx].P_CloseTheLoop := TRUE;
END_IF
nStartupSequence := nStartupSequence + 1;

14: // Check for track plausability (detect layout inconsistencies)
IF aTrackTable[nTrackIdx].CheckTrackTable() <> 0 THEN
  IF aTrackTable[nTrackIdx].P_TrackTableCheckResult THEN
    // Track layout valid
    nStartupSequence := nStartupSequence + 6;
  ELSE
    // Error in track layout
    nStartupSequence := nStartupSequence + 1;
  END_IF
END_IF

20: // Build track geometry command
IF fbTrackStandard[nTrackIdx].BuildTrackExt(aTrackTable[nTrackIdx]) THEN
  nStartupSequence := nStartupSequence + 1;
END_IF

21: // Building track
IF fbTrackStandard[nTrackIdx].P_BuildTrackExt.Done THEN
  nStartupSequence := nStartupSequence - 10;
ELSIF fbTrackStandard[nTrackIdx].P_BuildTrackExt.Error THEN
  nStartupSequence := nStartupSequence + 4;
END_IF

30: // Done
Startup := TRUE;
END_CASE

```

Syntax

```

CASE nStartupSequence OF
  0: // Init
    nStartupSequence := nStartupSequence + 1;

```

```

1:    // Assign mover references to the system
    FOR i := 1 TO moverCount DO
        fbXPlanarSystem.SetMoverStandard(i, fbXPlanarMover[i]);
    END_FOR
    nStartupSequence := nStartupSequence + 1;

2:    // Assign track references to the system
    FOR i := 1 TO maxTrackCount DO
        fbXPlanarSystem.SetTrackStandard(i, fbTrackStandard[i]);
    END_FOR
    nStartupSequence := nStartupSequence + 8;

10:   // Build track geometries
    nTrackIdx := 0;
    nStartupSequence := nStartupSequence + 1;

11:   // Start with an empty table
    nTrackIdx := nTrackIdx + 1;
    IF nTrackIdx > maxTrackCount THEN
        // All tracks are built
        nStartupSequence := nStartupSequence + 19;
    ELSE
        aTrackTable[nTrackIdx].ClearTrackTable();
        nStartupSequence := nStartupSequence + 1;
    END_IF

12:   // Fill track table with segments (qty of segments depends on each track)
    CASE nTrackIdx OF
        1:    // T1
            nStartFromTrack := 0;
            nEndAtTrack := 0;
            bCloseLoop := FALSE;

            aTrackTable[nTrackIdx].AddPoint(E_PointType.Point,
E_PointOption.none, 240, 120, 0);

            aTrackTable[nTrackIdx].AddPoint(E_PointType.Line_Start,
E_PointOption.none, 120, 240, 0);
            aTrackTable[nTrackIdx].AddPoint(E_PointType.Line_End,
E_PointOption.none, 120, 480, 0);

            aTrackTable[nTrackIdx].AddPoint(E_PointType.Point,
E_PointOption.none, 240, 600, 0);

        2:    // T2
            nStartFromTrack := 1;
            nEndAtTrack := 1;
            bCloseLoop := FALSE;

            aTrackTable[nTrackIdx].AddPoint(E_PointType.Line_Start,
E_PointOption.none, 360, 480, 0);

```

```

        aTrackTable[nTrackIdx].AddPoint(E_PointType.Line_End,
E_PointOption.none, 360, 240, 0);
        END_CASE
        nStartupSequence := nStartupSequence + 1;

13:    // Set additional design info
        IF nStartFromTrack > 0 THEN
            aTrackTable[nTrackIdx].P_StartFromTrack :=
fbTrackStandard[nStartFromTrack];
        END_IF
        IF nEndAtTrack > 0 THEN
            aTrackTable[nTrackIdx].P_EndAtTrack := fbTrackStandard[nEndAtTrack];
        END_IF
        IF bCloseLoop AND nStartFromTrack = 0 AND nEndAtTrack = 0 THEN
            aTrackTable[nTrackIdx].P_CloseTheLoop := TRUE;
        END_IF
        nStartupSequence := nStartupSequence + 1;

14:    // Check for track plausability (detect layout inconsistencies)
        IF aTrackTable[nTrackIdx].CheckTrackTable() <> 0 THEN
            IF aTrackTable[nTrackIdx].P_TrackTableCheckResult THEN
                // Track layout valid
                nStartupSequence := nStartupSequence + 6;
            ELSE
                // Error in track layout
                nStartupSequence := nStartupSequence + 1;
            END_IF
        END_IF

20:    // Build track geometry command
        IF fbTrackStandard[nTrackIdx].BuildTrackExt(aTrackTable[nTrackIdx]) THEN
            nStartupSequence := nStartupSequence + 1;
        END_IF

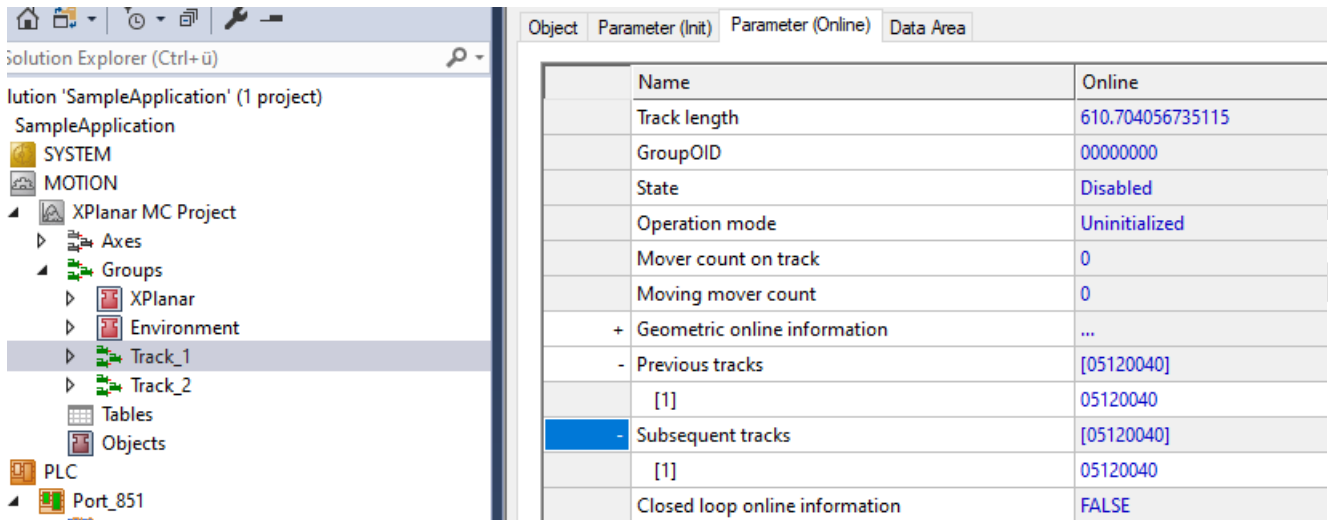
21:    // Building track
        IF fbTrackStandard[nTrackIdx].P_BuildTrackExt.Done THEN
            nStartupSequence := nStartupSequence - 10;
        ELSIF fbTrackStandard[nTrackIdx].P_BuildTrackExt.Error THEN
            nStartupSequence := nStartupSequence + 4;
        END_IF

30:    // Done
        Startup := TRUE;
END_CASE

```

Build the application, then reactivate the configuration and reload the project in the PLC.

After all geometries are built, the track info is shown in the Parameter (Online) tab of each track's object of the MOTION section. Track length and previous/subsequent tracks info are now available.



The screenshot shows the Beckhoff XPlanar software interface. On the left is the 'Solution Explorer (Ctrl+U)' pane, which displays a project tree. The tree is expanded to show the 'MOTION' section, specifically the 'XPlanar MC Project' and its 'Groups'. Under 'Groups', 'Track_1' is selected. The right pane shows the 'Parameter (Online)' tab for the selected track. The table below represents the data shown in this tab.

Object	Parameter (Init)	Parameter (Online)	Data Area
	Name	Online	
	Track length	610.704056735115	
	GroupOID	00000000	
	State	Disabled	
	Operation mode	Uninitialized	
	Mover count on track	0	
	Moving mover count	0	
	+ Geometric online information	...	
	- Previous tracks	[05120040]	
	[1]	05120040	
	- Subsequent tracks	[05120040]	
	[1]	05120040	
	Closed loop online information	FALSE	

Including tracks into Collision Avoidance

Once the tracks exist and their layouts have been verified, the Starting sequence can be modified to include them in the Collision Avoidance monitoring, like done with the movers.

```

IF bReset THEN
    Starting := FALSE;
    nStartingSequence := 0;
END_IF

CASE nStartingSequence OF
    0: // Init
        Execute(bReset:= TRUE);
        IF NOT bReset THEN
            nStartingSequence := nStartingSequence + 10;
        END_IF

    10: // Enable C.A. group command
        IF fbXPlanarSystem.EnableGroup() THEN
            nStartingSequence := nStartingSequence + 1;
        END_IF

    11: // Enabling group
        IF fbXPlanarSystem.P_EnableGroup.Done THEN
            nStartingSequence := nStartingSequence + 9;
        ELSIF fbXPlanarSystem.P_EnableGroup.Error THEN
            nStartingSequence := nStartingSequence + 4;
        END_IF

    20: // Add environment (surface boundaries) to group command
        IF fbXPlanarSystem.AddEnvironmentToGroup() THEN
            nStartingSequence := nStartingSequence + 1;
        END_IF

    21: // Adding environment to group
        IF fbXPlanarSystem.P_AddEnvironmentToGroup.Done THEN
            nStartingSequence := nStartingSequence + 9;
        ELSIF fbXPlanarSystem.P_AddEnvironmentToGroup.Error THEN
            nStartingSequence := nStartingSequence + 4;
        END_IF

    30: // Enable tracks command
        IF fbXPlanarSystem.EnableTracks() THEN
            nStartingSequence := nStartingSequence + 1;
        END_IF

    31: // Enabling tracks
        IF fbXPlanarSystem.P_EnableTracks.Done THEN
            nStartingSequence := nStartingSequence + 9;
        ELSIF fbXPlanarSystem.P_EnableTracks.Error THEN
            nStartingSequence := nStartingSequence + 4;
        END_IF

```

Modify the Starting sequence as shown to perform the additional preparation tasks.

- Collision Avoidance Group gets enabled
- The stator surface boundaries (Environment) object gets included in the group
- Tracks get enabled
- Tracks get included in the group
- Movers get enabled
- Movers get included in the group

The order of operations in the Starting sequence is important and must be observed.

```

40: // Add tracks to group command
   IF fbXPlanarSystem.AddTracksToGroup() THEN
       nStartingSequence := nStartingSequence + 1;
   END_IF

41: // Adding tracks to group
   IF fbXPlanarSystem.P_AddTracksToGroup.Done THEN
       nStartingSequence := nStartingSequence + 9;
   ELSIF fbXPlanarSystem.P_AddTracksToGroup.Error THEN
       nStartingSequence := nStartingSequence + 4;
   END_IF

50: // Enable movers command
   IF fbXPlanarSystem.EnableMovers() THEN
       nStartingSequence := nStartingSequence + 1;
   END_IF

51: // Enabling movers
   IF fbXPlanarSystem.P_EnableMovers.Done THEN
       nStartingSequence := nStartingSequence + 9;
   ELSIF fbXPlanarSystem.P_EnableTracks.Error THEN
       nStartingSequence := nStartingSequence + 4;
   END_IF

60: // Add movers to group command
   IF fbXPlanarSystem.AddMoversToGroup() THEN
       nStartingSequence := nStartingSequence + 1;
   END_IF

61: // Adding movers to group
   IF fbXPlanarSystem.P_AddMoversToGroup.Done THEN
       nStartingSequence := nStartingSequence + 19;
   ELSIF fbXPlanarSystem.P_AddMoversToGroup.Error THEN
       nStartingSequence := nStartingSequence + 4;
   END_IF

80: // Done
   Starting := TRUE;
END_CASE

```

Syntax

```

IF bReset THEN
    Starting := FALSE;
    nStartingSequence := 0;
END_IF

CASE nStartingSequence OF
    0:    // Init
        Execute(bReset:= TRUE);
        IF NOT bReset THEN
            nStartingSequence := nStartingSequence + 10;
        END_IF

    10:   // Enable C.A. group command
        IF fbXPlanarSystem.EnableGroup() THEN
            nStartingSequence := nStartingSequence + 1;
        END_IF

    11:   // Enabling group
        IF fbXPlanarSystem.P_EnableGroup.Done THEN
            nStartingSequence := nStartingSequence + 9;

```

```

        ELSIF fbXPlanarSystem.P_EnableGroup.Error THEN
            nStartingSequence := nStartingSequence + 4;
        END_IF

20:    // Add environment (surface boundaries) to group command
    IF fbXPlanarSystem.AddEnvironmentToGroup() THEN
        nStartingSequence := nStartingSequence + 1;
    END_IF

21:    // Adding environment to group
    IF fbXPlanarSystem.P_AddEnviromentToGroup.Done THEN
        nStartingSequence := nStartingSequence + 9;
    ELSIF fbXPlanarSystem.P_AddEnviromentToGroup.Error THEN
        nStartingSequence := nStartingSequence + 4;
    END_IF

30:    // Enable tracks command
    IF fbXPlanarSystem.EnableTracks() THEN
        nStartingSequence := nStartingSequence + 1;
    END_IF

31:    // Enabling tracks
    IF fbXPlanarSystem.P_EnableTracks.Done THEN
        nStartingSequence := nStartingSequence + 9;
    ELSIF fbXPlanarSystem.P_EnableTracks.Error THEN
        nStartingSequence := nStartingSequence + 4;
    END_IF

40:    // Add tracks to group command
    IF fbXPlanarSystem.AddTracksToGroup() THEN
        nStartingSequence := nStartingSequence + 1;
    END_IF

41:    // Adding tracks to group
    IF fbXPlanarSystem.P_AddTracksToGroup.Done THEN
        nStartingSequence := nStartingSequence + 9;
    ELSIF fbXPlanarSystem.P_AddTracksToGroup.Error THEN
        nStartingSequence := nStartingSequence + 4;
    END_IF

50:    // Enable movers command
    IF fbXPlanarSystem.EnableMovers() THEN
        nStartingSequence := nStartingSequence + 1;
    END_IF

51:    // Enabling movers
    IF fbXPlanarSystem.P_EnableMovers.Done THEN
        nStartingSequence := nStartingSequence + 9;
    ELSIF fbXPlanarSystem.P_EnableTracks.Error THEN
        nStartingSequence := nStartingSequence + 4;
    END_IF

60:    // Add movers to group command

```

```

        IF fbXPlanarSystem.AddMoversToGroup() THEN
            nStartingSequence := nStartingSequence + 1;
        END_IF

61:    // Adding movers to group
        IF fbXPlanarSystem.P_AddMoversToGroup.Done THEN
            nStartingSequence := nStartingSequence + 19;
        ELSIF fbXPlanarSystem.P_AddMoversToGroup.Error THEN
            nStartingSequence := nStartingSequence + 4;
        END_IF

80:    // Done
        Starting := TRUE;

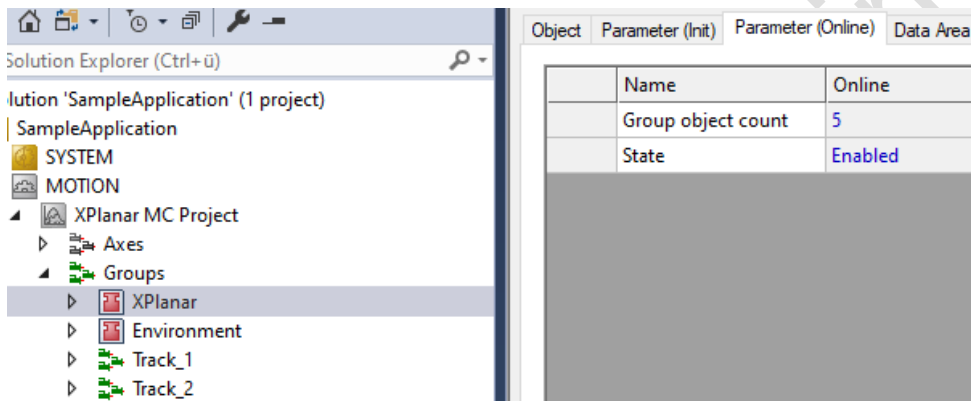
END_CASE

```

As the example shows it, tracks must be enabled (like movers) before they can be added to the C.A. group.

Build the application, reactivate the configuration, and reload the project in the PLC.

The Collision Avoidance group should now contain 5 elements after the Starting sequence has completed.



These 5 elements are:

- ▶ Environment (surface boundaries)
- ▶ 2 Movers
- ▶ 2 Tracks

Moving on tracks

Before they can be moved on tracks, movers must first “jump” on a track. Imagine a train without rails. Traveling would be quite hazardous...

This process of jumping on a specified track is called “joining” a track. To do so, the mover object provides a command called **JoinTrack**, to which a track is specified. The mover will then attempt to move towards the nearest point of that track and clip on it. From then on, the mover’s command mode changes from **FreeMovement** to **OnTrack**, signaling that the mover can be moved along the track (or track network). No further movement to any X-Y position can be performed until the mover leaves its track, via the command **LeaveTrack**, to which a target position is specified.

An important step has been skipped so far, that had no impact since the project is run in simulation mode and the movers are located at the same positions each time the application gets started. For any kind of motion axis, the application developer should consider a homing procedure before starting to move the axis around.

To implement this sort of homing, add a method named `MoverRecovery` with a return type `BOOL` to `FB_MotionSystem`.

As this method is going to be a sequence, add a state machine variable **nRecoverySequence** of type `UDINT` in the fb's declaration part. This variable will represent the current step in the sequence and can eventually be used to monitor the system's behavior.

When the method returns `TRUE`, it means the sequence completed successfully.

```

METHOD PRIVATE MoverRecovery : BOOL
VAR_INPUT
    bReset          : BOOL;    // Reset sequence
END_VAR
VAR
    i               : UDINT;
END_VAR
VAR_INST
    nTargetTrack    : ARRAY[1..maxNumMover] OF UDINT;
    stOptionsJoinTrack : ST_JoinTrackOptions;
    bMoversJoined    : BOOL;
    bMoverError      : BOOL;
END_VAR

```

An input parameter `bReset` will allow the sequence to be reset by the application.

In this example application, the mover recovery (homing) process consists solely of joining the closest track regarding each mover's initial position. Depending on the application's requirements, the mover recovery can become increasingly complex and must be considered with attention.

Due to mover initial positions being located on the tracks, there will be no movement during the JoinTrack execution.

```

IF bReset THEN
    MoverRecovery := FALSE;
    nRecoverySequence := 0;
END_IF

CASE nRecoverySequence OF
    0: // Init
        bMoversJoined := FALSE;
        bMoverError := FALSE;
        IF NOT bReset THEN
            nRecoverySequence := nRecoverySequence + 10;
        END_IF

    10: // Join tracks from initial positions
        nTargetTrack[1] := 1;
        nTargetTrack[2] := 2;
        stOptionsJoinTrack.useOrientation := TRUE;
        FOR i := 1 TO moverCount DO
            fbXPlanarMover[i].JoinTrack('Join track', fbTrackStandard[nTargetTrack[i]], 250, 1250, 12500, stOptionsJoinTrack);
        END_FOR
        nRecoverySequence := nRecoverySequence + 1;

    11: // Joining tracks
        bMoversJoined := TRUE;
        bMoverError := FALSE;
        FOR i := 1 TO moverCount DO
            bMoversJoined := bMoversJoined AND fbXPlanarMover[i].P_JoinTrack.Done;
            bMoverError := bMoverError OR fbXPlanarMover[i].P_JoinTrack.Error;
        END_FOR
        IF bMoversJoined THEN
            nRecoverySequence := nRecoverySequence + 9;
        ELSIF bMoverError THEN
            nRecoverySequence := nRecoverySequence + 4;
        END_IF

    20: // Done
        MoverRecovery := TRUE;
END_CASE

```

Syntax

```

METHOD PRIVATE MoverRecovery : BOOL
VAR_INPUT
    bReset          : BOOL;          // Reset sequence
END_VAR
VAR
    i               : UDINT;
END_VAR
VAR_INST
    nTargetTrack    : ARRAY[1..maxNumMover] OF UDINT;
    stOptionsJoinTrack : ST_JoinTrackOptions;
    bMoversJoined    : BOOL;
    bMoverError      : BOOL;
END_VAR
IF bReset THEN
    MoverRecovery := FALSE;
    nRecoverySequence := 0;
END_IF

CASE nRecoverySequence OF

```

```

0:    // Init
      bMoversJoined := FALSE;
      bMoverError := FALSE;
      IF NOT bReset THEN
        nRecoverySequence := nRecoverySequence + 10;
      END_IF

10:   // Join tracks from initial positions
      nTargetTrack[1] := 1;
      nTargetTrack[2] := 2;
      stOptionsJoinTrack.useOrientation := TRUE;
      FOR i := 1 TO moverCount DO
        fbXPlanarMover[i].JoinTrack('Join track',
fbTrackStandard[nTargetTrack[i]], 250, 1250, 12500, stOptionsJoinTrack);
      END_FOR
      nRecoverySequence := nRecoverySequence + 1;

11:   // Joining tracks
      bMoversJoined := TRUE;
      bMoverError := FALSE;
      FOR i := 1 TO moverCount DO
        bMoversJoined := bMoversJoined AND
fbXPlanarMover[i].P_JoinTrack.Done;
        bMoverError := bMoverError OR fbXPlanarMover[i].P_JoinTrack.Error;
      END_FOR
      IF bMoversJoined THEN
        nRecoverySequence := nRecoverySequence + 9;
      ELSIF bMoverError THEN
        nRecoverySequence := nRecoverySequence + 4;
      END_IF

20:   // Done
      MoverRecovery := TRUE;
END_CASE

```

Hint

To get access to the additional required data types, add the **Tc3_Mc3PlanarMotion** lib in the project's references, as already done with Tc3_Physics.

The MoverRecovery method will then be called as last step of the Starting sequence, ensuring movers are “homed” (system in a defined state) and standing on tracks before they can be moved around the track network designed in this example.

```

CASE nStartingSequence OF
  0: // Init
    Execute(bReset:= TRUE);
    MoverRecovery(bReset:= TRUE);
    IF NOT bReset THEN
      nStartingSequence := nStartingSequence + 10;
    END_IF

  10: // Enable C.A. group command
    IF fbXPlanarSystem.EnableGroup() THEN
      nStartingSequence := nStartingSequence + 1;
    END_IF

  61: // Adding movers to group
    IF fbXPlanarSystem.P_AddMoversToGroup.Done THEN
      nStartingSequence := nStartingSequence + 9;
    ELSIF fbXPlanarSystem.P_AddMoversToGroup.Error THEN
      nStartingSequence := nStartingSequence + 4;
    END_IF

  70: // Mover recovery (application specific homing)
    IF MoverRecovery(bReset:= FALSE) THEN
      nStartingSequence := nStartingSequence + 10;
    END_IF

  80: // Done
    Starting := TRUE;
END_CASE

```

Reset the MoverRecovery sequence at the beginning of the Starting sequence to ensure it will be executed from its beginning.

Execute the MoverRecovery sequence as final step of the Starting sequence.

Syntax

```

IF bReset THEN
  Starting := FALSE;
  nStartingSequence := 0;
END_IF

CASE nStartingSequence OF
  0: // Init
    Execute(bReset:= TRUE);
    MoverRecovery(bReset:= TRUE);
    IF NOT bReset THEN
      nStartingSequence := nStartingSequence + 10;
    END_IF

  10: // Enable C.A. group command
    IF fbXPlanarSystem.EnableGroup() THEN
      nStartingSequence := nStartingSequence + 1;
    END_IF

  11: // Enabling group
    IF fbXPlanarSystem.P_EnableGroup.Done THEN
      nStartingSequence := nStartingSequence + 9;
    ELSIF fbXPlanarSystem.P_EnableGroup.Error THEN

```

```

        nStartingSequence := nStartingSequence + 4;
    END_IF

20:    // Add environment (surface boundaries) to group command
    IF fbXPlanarSystem.AddEnvironmentToGroup() THEN
        nStartingSequence := nStartingSequence + 1;
    END_IF

21:    // Adding environment to group
    IF fbXPlanarSystem.P_AddEnviromentToGroup.Done THEN
        nStartingSequence := nStartingSequence + 9;
    ELSIF fbXPlanarSystem.P_AddEnviromentToGroup.Error THEN
        nStartingSequence := nStartingSequence + 4;
    END_IF

30:    // Enable tracks command
    IF fbXPlanarSystem.EnableTracks() THEN
        nStartingSequence := nStartingSequence + 1;
    END_IF

31:    // Enabling tracks
    IF fbXPlanarSystem.P_EnableTracks.Done THEN
        nStartingSequence := nStartingSequence + 9;
    ELSIF fbXPlanarSystem.P_EnableTracks.Error THEN
        nStartingSequence := nStartingSequence + 4;
    END_IF

40:    // Add tracks to group command
    IF fbXPlanarSystem.AddTracksToGroup() THEN
        nStartingSequence := nStartingSequence + 1;
    END_IF

41:    // Adding tracks to group
    IF fbXPlanarSystem.P_AddTracksToGroup.Done THEN
        nStartingSequence := nStartingSequence + 9;
    ELSIF fbXPlanarSystem.P_AddTracksToGroup.Error THEN
        nStartingSequence := nStartingSequence + 4;
    END_IF

50:    // Enable movers command
    IF fbXPlanarSystem.EnableMovers() THEN
        nStartingSequence := nStartingSequence + 1;
    END_IF

51:    // Enabling movers
    IF fbXPlanarSystem.P_EnableMovers.Done THEN
        nStartingSequence := nStartingSequence + 9;
    ELSIF fbXPlanarSystem.P_EnableTracks.Error THEN
        nStartingSequence := nStartingSequence + 4;
    END_IF

60:    // Add movers to group command
    IF fbXPlanarSystem.AddMoversToGroup() THEN

```

```

        nStartingSequence := nStartingSequence + 1;
    END_IF

61:    // Adding movers to group
    IF fbXPlanarSystem.P_AddMoversToGroup.Done THEN
        nStartingSequence := nStartingSequence + 9;
    ELSIF fbXPlanarSystem.P_AddMoversToGroup.Error THEN
        nStartingSequence := nStartingSequence + 4;
    END_IF

70:    // Mover recovery (application specific homing)
    IF MoverRecovery(bReset:= FALSE) THEN
        nStartingSequence := nStartingSequence + 10;
    END_IF

80:    // Done
    Starting := TRUE;

END_CASE

```

Rebuild the application, reactivate the configuration, and reload the project in the PLC.

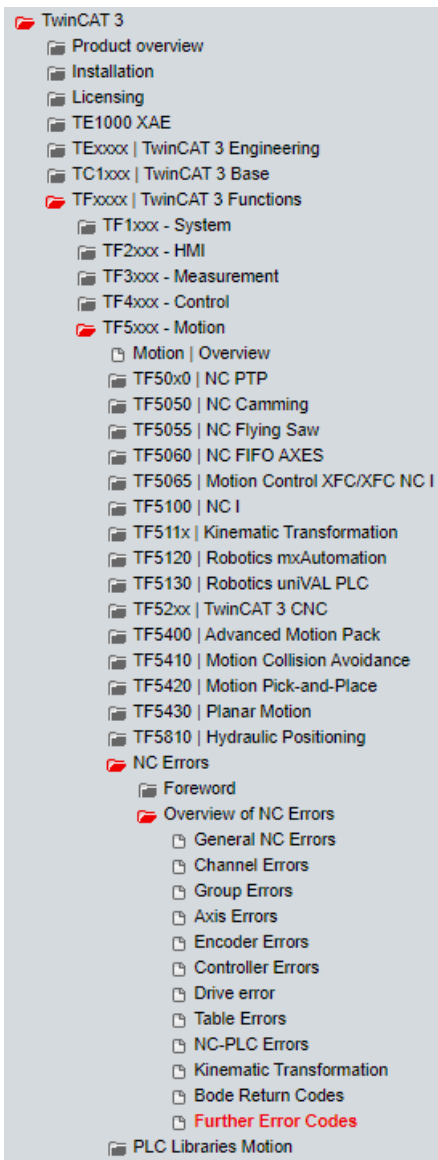
After successfully starting, the application can no longer run the Execute, due to the Command Mode of the movers being **OnTrack** rather than **FreeMovement**.

fbXMotionUnit	FB_MotionSystem	
fbXPlanarSystem	FB_XPlanarSystem	
fbXPlanarMover	ARRAY [1..maxNumMover] OF FB_XPlanarMoverStandard	
fbTrackStandard	ARRAY [1..maxTrackCount] OF FB_TrackStandard	
nStartupSequence	UDINT	30
nStartingSequence	UDINT	80
nRecoverySequence	UDINT	20
nExecutingSequence	UDINT	0
nState	UDINT	40

In the Execute sequence, if the command mode was disregarded and a MoveToPosition command was executed (it can be done by writing nExecutingSequence = 10 while online), the command would return an error and no movement occur. The Execute sequence immediately goes to step 15 (error handling).

fbXPlanarMover[2]	FB_XPlanarMoverStandard	
stMoverRef	MC_PlanarMover	
nTrackId	UINT	0
nId	UINT	0
enableCmdFB	MC_PlanarFeedback	
stRtEnable	ST_CommandReturn	
disableCmdFB	MC_PlanarFeedback	
stRtDisable	ST_CommandReturn	
joinTrackCmdFB	MC_PlanarFeedback	
stRtJoinTrack	ST_CommandReturn	
leaveTrackCmdFB	MC_PlanarFeedback	
stRtLeaveTrack	ST_CommandReturn	
moveOnTrackCmdFB	MC_PlanarFeedback	
stRtMoveOnTrack	ST_CommandReturn	
moveToPosCmdFB	MC_PlanarFeedback	
Active	BOOL	FALSE
Busy	BOOL	FALSE
Done	BOOL	FALSE
Aborted	BOOL	FALSE
Error	BOOL	TRUE
ErrorId	UDINT	33105
objectInfo	PlanarObjectInfo	

The error code meaning can be found in the Information System under:



Most of the XPlanar commands errors are listed in **Further Error Codes** range, but not exclusively. Please check other error code ranges if required.

Our example error code 33105 mentions that the MoveToPosition command was prohibited since the Command Mode value of the mover is currently OnTrack. This means the mover should now be commanded via MoveOnTrack commands instead.

Modify the Execute sequence to move on track.

The previous sequence steps remain unchanged, to allow testing in both modes if required (by skipping the MoverRecovery call in the Starting sequence).

```

METHOD Execute : BOOL
VAR_INPUT
    bReset          : BOOL; // Reset sequence
END_VAR
VAR
    i               : UDINT;
END_VAR
VAR_INST
    targetPos       : ARRAY[1..maxNumMover] OF PositionXYC;
    bMoversInPos    : BOOL;
    bMoverError      : BOOL;

    fTargetPosOnTrack : ARRAY[1..maxNumMover] OF LREAL;
END_VAR

IF bReset THEN
    Execute := FALSE;
    nExecutingSequence := 0;
END_IF

CASE nExecutingSequence OF
    0: // Init
        bMoversInPos := FALSE;
        bMoverError := FALSE;
        IF NOT bReset THEN
            IF fbXPlanarMover[1].P_CommandMode = MC_PLANAR_MOVER_COMMAND_MODE.FreeMovement THEN
                nExecutingSequence := nExecutingSequence + 10;
            ELSIF fbXPlanarMover[1].P_CommandMode = MC_PLANAR_MOVER_COMMAND_MODE.OnTrack THEN
                nExecutingSequence := nExecutingSequence + 40;
            END_IF
        END_IF

// ----- A : Without MoverRecovery (FreeMovement) -----
    10: // Move Mover 1 to X 120 mm Y 120 mm and Mover 2 to X 360 mm Y 600 mm with 500 mm/s
        targetPos[1].SetValuesXYC(120, 120, 0);
        targetPos[2].SetValuesXYC(360, 600, 0);
        FOR i := 1 TO moverCount DO
            fbXPlanarMover[i].MoveToPosition('Move to XY', targetPos[i], 500, 1250, 1250, 12500);
        END_FOR
        nExecutingSequence := nExecutingSequence + 1;

    11: // Movers moving
        bMoversInPos := TRUE;
        bMoverError := FALSE;
        FOR i := 1 TO moverCount DO
            bMoversInPos := bMoversInPos AND fbXPlanarMover[i].P_MoveToPosition.Done;
            bMoverError := bMoverError OR fbXPlanarMover[i].P_MoveToPosition.Error;
        END_FOR
        IF bMoversInPos THEN
            nExecutingSequence := nExecutingSequence + 9;
        ELSIF bMoverError THEN
            nExecutingSequence := nExecutingSequence + 4;
        END_IF

    20: // Move Mover 1 to X 120 mm Y 600 mm and Mover 2 to X 360 mm Y 120 mm with 500 mm/s
        targetPos[1].SetValuesXYC(120, 600, 0);
        targetPos[2].SetValuesXYC(360, 120, 0);
        FOR i := 1 TO moverCount DO
            fbXPlanarMover[i].MoveToPosition('Move to XY', targetPos[i], 500, 1250, 1250, 12500);
        END_FOR
        nExecutingSequence := nExecutingSequence + 1;

```

```

21: // Movers moving
bMoversInPos := TRUE;
bMoverError := FALSE;
FOR i := 1 TO moverCount DO
    bMoversInPos := bMoversInPos AND fbXPlanarMover[i].P_MoveToPosition.Done;
    bMoverError := bMoverError OR fbXPlanarMover[i].P_MoveToPosition.Error;
END FOR
IF bMoversInPos THEN
    nExecutingSequence := nExecutingSequence + 9;
ELSIF bMoverError THEN
    nExecutingSequence := nExecutingSequence + 4;
END_IF

30: // Done - Loop
Execute := FALSE; // Operator must abort manually (via Stop or E-Stop)
nExecutingSequence := nExecutingSequence - 20;

// ----- B : With MoverRecovery (OnTrack) -----
40: // Move Mover 1 to end of track 1 (top) and Mover 2 to end of track 2 (bottom)
fTargetPosOnTrack[1] := fbTrackStandard[1].GetLength();
fTargetPosOnTrack[2] := fbTrackStandard[2].GetLength();
FOR i := 1 TO moverCount DO
    fbXPlanarMover[i].MoveOnTrack('Move on track', fbTrackStandard[i], fTargetPosOnTrack[i], 500, 1250, 12500, 230, MC_DIRECTION.mcDirectionPositive);
END FOR
nExecutingSequence := nExecutingSequence + 1;

41: // Movers moving on track
bMoversInPos := TRUE;
bMoverError := FALSE;
FOR i := 1 TO moverCount DO
    bMoversInPos := bMoversInPos AND fbXPlanarMover[i].P_MoveOnTrack.Done;
    bMoverError := bMoverError OR fbXPlanarMover[i].P_MoveOnTrack.Error;
END FOR
IF bMoversInPos THEN
    nExecutingSequence := nExecutingSequence + 9;
ELSIF bMoverError THEN
    nExecutingSequence := nExecutingSequence + 4;
END_IF

50: // Move Mover 2 queuing behind Mover 1
fTargetPosOnTrack[2] := fTargetPosOnTrack[1];
fbXPlanarMover[2].MoveOnTrack('Move to cue', fbTrackStandard[1], fTargetPosOnTrack[2], 500, 1250, 12500, 230, MC_DIRECTION.mcDirectionPositive);
nExecutingSequence := nExecutingSequence + 1;

51: // Mover 2 moving to queue
IF fbXPlanarMover[2].P_MoveOnTrack.Active AND fbXPlanarMover[2].P_IsQueuing THEN
    nExecutingSequence := nExecutingSequence + 9;
ELSIF fbXPlanarMover[2].P_MoveOnTrack.Error THEN
    nExecutingSequence := nExecutingSequence + 4;
END_IF

60: // Move Mover 1 to end of track 2 (bottom)
fTargetPosOnTrack[1] := fbTrackStandard[2].GetLength();
fbXPlanarMover[1].MoveOnTrack('Move on track', fbTrackStandard[2], fTargetPosOnTrack[1], 500, 1250, 12500, 230, MC_DIRECTION.mcDirectionPositive);
nExecutingSequence := nExecutingSequence + 1;

```

```

61: // Movers moving on track
bMoversInPos := TRUE;
bMoverError := FALSE;
FOR i := 1 TO moverCount DO
    bMoversInPos := bMoversInPos AND fbXPlanarMover[i].P_MoveOnTrack.Done;
    bMoverError := bMoverError OR fbXPlanarMover[i].P_MoveOnTrack.Error;
END_FOR
IF bMoversInPos THEN
    nExecutingSequence := nExecutingSequence + 9;
ELSIF bMoverError THEN
    nExecutingSequence := nExecutingSequence + 4;
END_IF

70: // Move Mover 1 to X 120 mm Y 360 mm (on track 1) and Mover 2 to X 360 mm Y 360 mm (on track 2)
targetPos[1].SetValuesXYC(120, 360, 0);
targetPos[2].SetValuesXYC(360, 360, 0);
FOR i := 1 TO moverCount DO
    fTargetPosOnTrack[i] := fbTrackStandard[i].GetTrackPositionAt(targetPos[i]);
    fbXPlanarMover[i].MoveOnTrack('Move on track', fbTrackStandard[i], fTargetPosOnTrack[i], 500, 1250, 12500, 230, MC_DIRECTION.mcDirectionPositive);
END_FOR
nExecutingSequence := nExecutingSequence + 1;

71: // Movers moving on track
bMoversInPos := TRUE;
bMoverError := FALSE;
FOR i := 1 TO moverCount DO
    bMoversInPos := bMoversInPos AND fbXPlanarMover[i].P_MoveOnTrack.Done;
    bMoverError := bMoverError OR fbXPlanarMover[i].P_MoveOnTrack.Error;
END_FOR
IF bMoversInPos THEN
    nExecutingSequence := nExecutingSequence + 9;
ELSIF bMoverError THEN
    nExecutingSequence := nExecutingSequence + 4;
END_IF

80: // Done - Loop
Execute := FALSE; // Operator must abort manually (via Stop or E-Stop)
nExecutingSequence := nExecutingSequence - 40;
END_CASE

```

Syntax

```

METHOD Execute : BOOL
VAR_INPUT
    bReset          : BOOL; // Reset sequence
END_VAR
VAR
    i               : UDINT;
END_VAR
VAR_INST
    targetPos       : ARRAY[1..maxNumMover] OF PositionXYC;
    bMoversInPos    : BOOL;
    bMoverError     : BOOL;

    fTargetPosOnTrack : ARRAY[1..maxNumMover] OF LREAL;
END_VAR
IF bReset THEN
    Execute := FALSE;
    nExecutingSequence := 0;
END_IF

CASE nExecutingSequence OF
    0: // Init
        bMoversInPos := FALSE;
        bMoverError := FALSE;
        IF NOT bReset THEN
            IF fbXPlanarMover[1].P_CommandMode =
MC_PLANAR_MOVER_COMMAND_MODE.FreeMovement THEN
                nExecutingSequence := nExecutingSequence + 10;

```

```

        ELSIF fbXPlanarMover[1].P_CommandMode =
MC_PLANAR_MOVER_COMMAND_MODE.OnTrack THEN
            nExecutingSequence := nExecutingSequence + 40;
        END_IF
    END_IF

// ----- A : Without MoverRecovery (FreeMovement) -----
10:    // Move Mover 1 to X 120 mm Y 120 mm and Mover 2 to X 360 mm Y 600 mm with
500 mm/s
        targetPos[1].SetValuesXYC(120, 120, 0);
        targetPos[2].SetValuesXYC(360, 600, 0);
        FOR i := 1 TO moverCount DO
            fbXPlanarMover[i].MoveToPosition('Move to XY', targetPos[i], 500,
1250, 1250, 12500);
        END_FOR
        nExecutingSequence := nExecutingSequence + 1;

11:    // Movers moving
        bMoversInPos := TRUE;
        bMoverError := FALSE;
        FOR i := 1 TO moverCount DO
            bMoversInPos := bMoversInPos AND
fbXPlanarMover[i].P_MoveToPosition.Done;
            bMoverError := bMoverError OR
fbXPlanarMover[i].P_MoveToPosition.Error;
        END_FOR
        IF bMoversInPos THEN
            nExecutingSequence := nExecutingSequence + 9;
        ELSIF bMoverError THEN
            nExecutingSequence := nExecutingSequence + 4;
        END_IF

20:    // Move Mover 1 to X 120 mm Y 600 mm and Mover 2 to X 360 mm Y 120 mm with
500 mm/s
        targetPos[1].SetValuesXYC(120, 600, 0);
        targetPos[2].SetValuesXYC(360, 120, 0);
        FOR i := 1 TO moverCount DO
            fbXPlanarMover[i].MoveToPosition('Move to XY', targetPos[i], 500,
1250, 1250, 12500);
        END_FOR
        nExecutingSequence := nExecutingSequence + 1;

21:    // Movers moving
        bMoversInPos := TRUE;
        bMoverError := FALSE;
        FOR i := 1 TO moverCount DO
            bMoversInPos := bMoversInPos AND
fbXPlanarMover[i].P_MoveToPosition.Done;
            bMoverError := bMoverError OR
fbXPlanarMover[i].P_MoveToPosition.Error;
        END_FOR
        IF bMoversInPos THEN
            nExecutingSequence := nExecutingSequence + 9;

```

```

        ELSIF bMoverError THEN
            nExecutingSequence := nExecutingSequence + 4;
        END_IF

30:    // Done - Loop
    Execute := FALSE; // Operator must abort manually (via Stop or E-Stop)
    nExecutingSequence := nExecutingSequence - 20;

// ----- B : With MoverRecovery (OnTrack) -----
40:    // Move Mover 1 to end of track 1 (top) and Mover 2 to end of track 2
(bottom)
    fTargetPosOnTrack[1] := fbTrackStandard[1].GetLength();
    fTargetPosOnTrack[2] := fbTrackStandard[2].GetLength();
    FOR i := 1 TO moverCount DO
        fbXPlanarMover[i].MoveOnTrack('Move on track', fbTrackStandard[i],
fTargetPosOnTrack[i], 500, 1250, 12500, 230, MC_DIRECTION.mcDirectionPositive);
    END_FOR
    nExecutingSequence := nExecutingSequence + 1;

41:    // Movers moving on track
    bMoversInPos := TRUE;
    bMoverError := FALSE;
    FOR i := 1 TO moverCount DO
        bMoversInPos := bMoversInPos AND
fbXPlanarMover[i].P_MoveOnTrack.Done;
        bMoverError := bMoverError OR fbXPlanarMover[i].P_MoveOnTrack.Error;
    END_FOR
    IF bMoversInPos THEN
        nExecutingSequence := nExecutingSequence + 9;
    ELSIF bMoverError THEN
        nExecutingSequence := nExecutingSequence + 4;
    END_IF

50:    // Move Mover 2 queueing behind Mover 1
    fTargetPosOnTrack[2] := fTargetPosOnTrack[1];
    fbXPlanarMover[2].MoveOnTrack('Move to cue', fbTrackStandard[1],
fTargetPosOnTrack[2], 500, 1250, 12500, 230, MC_DIRECTION.mcDirectionPositive);
    nExecutingSequence := nExecutingSequence + 1;

51:    // Mover 2 moving to queue
    IF fbXPlanarMover[2].P_MoveOnTrack.Active AND fbXPlanarMover[2].P_IsQueueing
THEN
        nExecutingSequence := nExecutingSequence + 9;
    ELSIF fbXPlanarMover[2].P_MoveOnTrack.Error THEN
        nExecutingSequence := nExecutingSequence + 4;
    END_IF

60:    // Move Mover 1 to end of track 2 (bottom)
    fTargetPosOnTrack[1] := fbTrackStandard[2].GetLength();
    fbXPlanarMover[1].MoveOnTrack('Move on track', fbTrackStandard[2],
fTargetPosOnTrack[1], 500, 1250, 12500, 230, MC_DIRECTION.mcDirectionPositive);
    nExecutingSequence := nExecutingSequence + 1;

```

```

61:    // Movers moving on track
    bMoversInPos := TRUE;
    bMoverError := FALSE;
    FOR i := 1 TO moverCount DO
        bMoversInPos := bMoversInPos AND
fbXPlanarMover[i].P_MoveOnTrack.Done;
        bMoverError := bMoverError OR fbXPlanarMover[i].P_MoveOnTrack.Error;
    END_FOR
    IF bMoversInPos THEN
        nExecutingSequence := nExecutingSequence + 9;
    ELSIF bMoverError THEN
        nExecutingSequence := nExecutingSequence + 4;
    END_IF

70:    // Move Mover 1 to X 120 mm Y 360 mm (on track 1) and Mover 2 to X 360 mm Y
360 mm (on track 2)
    targetPos[1].SetValuesXYC(120, 360, 0);
    targetPos[2].SetValuesXYC(360, 360, 0);
    FOR i := 1 TO moverCount DO
        fTargetPosOnTrack[i] :=
fbTrackStandard[i].GetTrackPositionAt(targetPos[i]);
        fbXPlanarMover[i].MoveOnTrack('Move on track', fbTrackStandard[i],
fTargetPosOnTrack[i], 500, 1250, 12500, 230, MC_DIRECTION.mcDirectionPositive);
    END_FOR
    nExecutingSequence := nExecutingSequence + 1;

71:    // Movers moving on track
    bMoversInPos := TRUE;
    bMoverError := FALSE;
    FOR i := 1 TO moverCount DO
        bMoversInPos := bMoversInPos AND
fbXPlanarMover[i].P_MoveOnTrack.Done;
        bMoverError := bMoverError OR fbXPlanarMover[i].P_MoveOnTrack.Error;
    END_FOR
    IF bMoversInPos THEN
        nExecutingSequence := nExecutingSequence + 9;
    ELSIF bMoverError THEN
        nExecutingSequence := nExecutingSequence + 4;
    END_IF

80:    // Done - Loop
    Execute := FALSE; // Operator must abort manually (via Stop or E-Stop)
    nExecutingSequence := nExecutingSequence - 40;

END_CASE

```

Rebuild the application, reactivate the configuration, and reload the project in the PLC.

It can be observed, once the Execute state has been reached by the application, that the nExecutingSequence is endlessly looping from steps 40 to 80, indicating the movements of the movers along the tracks.

fbXMotionUnit	FB_MotionSystem	
fbXPlanarSystem	FB_XPlanarSystem	
fbXPlanarMover	ARRAY [1..maxNumMover] OF FB_XPlanarMoverStandard	
fbTrackStandard	ARRAY [1..maxTrackCount] OF FB_TrackStandard	
nStartupSequence	UDINT	30
nStartingSequence	UDINT	80
nRecoverySequence	UDINT	20
nExecutingSequence	UDINT	71
nState	UDINT	40

```

1 CASE nState 40 OF
2   0: // Init
3     nState 40 := nState 40 + 1;
4
5   1: // Startup sequence
6     IF fbXMotionUnit.Startup() THEN
7       nState 40 := nState 40 + 29;
8     END_IF
9
10  30: // Preparing sequence
11    IF fbXMotionUnit.Starting(bReset:= FALSE) THEN
12      nState 40 := nState 40 + 10;
13    END_IF
14
15  40: // Executing sequence
16    fbXMotionUnit.Execute(bReset:= FALSE);
17 END_CASE
18
19 // Cyclic update
20 fbXMotionUnit.Cycle();

```

The XPlanar Configurator Live-View tab can be used to monitor the movements.

Expression	Type	Value
fbXMotionUnit	FB_MotionSystem	
fbXPlanarSystem	FB_XPlanarSystem	
fbXPlanarMover	ARRAY [1..maxNumMover] OF FB_XPlanarMoverStandard	
fbTrackStandard	ARRAY [1..maxTrackCount] OF FB_TrackStandard	
nStartupSequence	UDINT	30
nStartingSequence	UDINT	80
nRecoverySequence	UDINT	20
nExecutingSequence	UDINT	61
nState	UDINT	40

```

1 CASE nState 40 OF
2   0: // Init
3     nState 40 := nState 40 + 1;
4
5   1: // Startup sequence
6     IF fbXMotionUnit.Startup() THEN
7       nState 40 := nState 40 + 29;
8     END_IF
9
10  30: // Preparing sequence
11    IF fbXMotionUnit.Starting(bReset:= FALSE) THEN
12      nState 40 := nState 40 + 10;
13    END_IF
14
15  40: // Executing sequence
16    fbXMotionUnit.Execute(bReset:= FALSE);
17 END_CASE
18
19 // Cyclic update
20 fbXMotionUnit.Cycle();
21 RETURN

```

Observations and explanations

Queueing principle

Steps 50 - 51 of the Execute sequence introduce the principle of queueing with Mover 2.

Mover 2 is commanded to move to Mover 1's position, as shown by the values of fTargetPosOnTrack[1] and fTargetPosOnTrack[2].

fTargetPosOnTrack	ARRAY [1..maxNumMover] OF LREAL	
fTargetPosOnTrack[1]	LREAL	610.70405673511493
fTargetPosOnTrack[2]	LREAL	610.70405673511493


```

1  IF bReset ??? THEN [2 lines]
4  END_IF
5
6  CASE nExecutingSequence 51 OF
7    0: // Init [10 lines]
18  // ----- A : Without MoverRecovery (FreeMovement) ----- [46 lines]
65  // ----- B : With MoverRecovery (OnTrack) -----
66  40: // Move Mover 1 to end of track 1 (top) and Mover 2 to end of track 2 (bottom) [7 lines]
74  41: // Movers moving on track [12 lines]
87  50: // Move Mover 2 queueing behind Mover 1
88    fTargetPosOnTrack[2] := fTargetPosOnTrack[1];
89    fbXPlanarMover[2].MoveOnTrack('Move to cue', fbTrackStandard[1], fTargetPosOnTrack[2]
90    nExecutingSequence := nExecutingSequence + 1;
91
92  51: // Mover 2 moving to queue
93    IF fbXPlanarMover[2].P_MoveOnTrack.Active AND fbXPlanarMover[2].P_IsQueueing THEN
94      nExecutingSequence := nExecutingSequence + 9;
95    ELSIF fbXPlanarMover[2].P_MoveOnTrack.Error THEN
96      nExecutingSequence := nExecutingSequence + 4;
97    END_IF

```

The Collision Avoidance monitoring prevents Mover 2 from crashing into Mover 1 by maintaining a minimum gap (mover center to mover center) on the track path with its predecessor (Mover 1 in this case). The gap value is passed as parameter for the MoveOnTrack command and can thus be modified each time a mover is commanded.

As a result, Mover 2 doesn't reach its target and the command doesn't complete for as long as Mover 1 is standing there.

To prevent from getting stuck in the sequence, instead of waiting for the command to complete (.Done flag), the queueing situation can be observed with help of the P_IsQueueing property, as shown in step 51. This ensures that the mover has arrived close to its target and is standing still, while the command remains active.

The Busy indicators are still TRUE, meaning the command is not done yet. The mover's P_Busy property reflects these indicators state.

Mover[2]	FB_XPlanarMoverStandard	
stMoverRef	MC_PlanarMover	
PLCTOMC	CDT_PLCTOMC_PLANAR_MOVER	
MCTOPLC	CDT_MCTOPLC_PLANAR_MOVER	
STD	REFERENCE TO CDT_MCTOPLC_PLANAR_MOVER_STD	
MoverOID	OTCID	16#05110020
GroupOID	OTCID	16#05120010
State	MC_PLANAR_STATE	Enabled
CommandMode	MC_PLANAR_MOVER_COMMAND_MODE	OnTrack
Busy	MoverBusy	
busyMover	BIT	TRUE
busyXYC	BIT	TRUE
busyX	BIT	TRUE
busyY	BIT	TRUE
busyZ	BIT	FALSE
busyA	BIT	FALSE
busyB	BIT	FALSE
busyC	BIT	TRUE

When Mover 1 is commanded to go away, Mover 2 is completing its commanded movement, reaching its target.

Thus, step 61 of the Execute sequence monitors the arrival of both movers, even though step 60 commanded Mover 1 only.

```

60: // Move Mover 1 to end of track 2 (bottom)
fTargetPosOnTrack[1] := fbTrackStandard[2].GetLength();
fbXPlanarMover[1].MoveOnTrack('Move on track', fbTrackStandard[2], fTargetPosOnTrack[1]
nExecutingSequence := nExecutingSequence + 1;

61: // Movers moving on track
bMoversInPos := TRUE;
bMoverError := FALSE;
FOR i := 1 TO moverCount DO
    bMoversInPos := bMoversInPos AND fbXPlanarMover[i].P_MoveOnTrack.Done;
    bMoverError := bMoverError OR fbXPlanarMover[i].P_MoveOnTrack.Error;
END FOR
IF bMoversInPos THEN
    nExecutingSequence := nExecutingSequence + 9;
ELSIF bMoverError THEN
    nExecutingSequence := nExecutingSequence + 4;
END_IF

```

Track networks

The Execute sequence shows how tracks forming a network can be used almost like a single track. Movers are commanded to move to their target positions, without the need of leaving and joining each individual track every single time. The actual track on which the target position resides must be passed to each command.

By looking at a track's info, the number of movers located on it (MoverCountOnTrack) as well as its state and operation mode can be observed.

fbTrackStandard[1]	FB_TrackStandard	
Busy	BOOL	FALSE
stTrackRef	MC_PlanarTrack	
MCTOPLC_STD	CDT_MCTOPLC_PLANAR_TRACK	
TrackOID	OTCID	16#05120030
GroupOID	OTCID	16#05120010
State	MC_PLANAR_STATE	Enabled
OperationMode	MC_PLANAR_TRACK_OPERATION_MODE	Moving
MoverCountOnTrack	UDINT	1
MovingMoverCount	UDINT	1
Error	BOOL	FALSE
ErrorId	UDINT	0

Some restrictions apply to track networks.

- ▶ A mover can pass from a track to its successor in a network in positive direction but is not allowed to move across several tracks in negative direction. Backwards movements are possible within a track only.
- ▶ Modulo movements cannot be performed on a track network (e.g., additional turns, shortest way with tolerance window, ...). To obtain similar movements, intermediate target positions must be considered using several commands, without waiting for completion of the intermediate commands.

Geometric relationships

Steps 70 - 71 of the Execute sequence show how geometric positions on the tiles surface can be transformed into positions on a track and vice-versa.

Moving along tracks is a one-dimension operation, even though tracks are designed anywhere over the surface. Thus, to reach a specific geometric position (which must lie along the track path), the transform method GetTrackPositionAt is used.

The opposite conversion is also possible by using the GetMoverPositionAt transform method.

```

70: // Move Mover 1 to X 120 mm Y 360 mm (on track 1) and Mover 2 to X 360 mm Y 360 mm (on track 2)
targetPos[1].SetValuesXYC(120, 360, 0);
targetPos[2].SetValuesXYC(360, 360, 0);
FOR i := 1 TO moverCount DO
    fTargetPosOnTrack[i] := fbTrackStandard[i].GetTrackPositionAt(targetPos[i]);
    fbXPlanarMover[i].MoveOnTrack('Move on track', fbTrackStandard[i], fTargetPosOnTrack[i], 500,
END_FOR
nExecutingSequence := nExecutingSequence + 1;

71: // Movers moving on track
bMoversInPos := TRUE;
bMoverError := FALSE;
FOR i := 1 TO moverCount DO
    bMoversInPos := bMoversInPos AND fbXPlanarMover[i].P_MoveOnTrack.Done;
    bMoverError := bMoverError OR fbXPlanarMover[i].P_MoveOnTrack.Error;
END_FOR
IF bMoversInPos THEN
    nExecutingSequence := nExecutingSequence + 9;
ELSIF bMoverError THEN
    nExecutingSequence := nExecutingSequence + 4;
END_IF

```

Tc3_XPlanarStandard API

Data Types

Enums

E_HaltDynamicsMode

Type of movements to which the dynamics apply. It can be either single coordinate movements or track (path) movements.

Syntax

```
TYPE E_HaltDynamicsMode :
(
    AxisX_AxisY_CoordinatedOnPath := 0,
    AxisX_AxisY_Separately        := 1
);
END_TYPE
```

Values

Name	Description
AxisX_AxisY_CoordinatedOnPath	Dynamics constraint relevant for MoveOnTrack commands
AxisX_AxisY_Separately	Dynamics constraint relevant for individual coordinate movements (MoveToPosition, MoveC, ...)

E_MoverCommandType

Command currently executed (internal handling by the mover object).

Syntax

```
TYPE E_MoverCommandType :
(
    eMoverCommand_eNoCommand           := 0,
    eMoverCommand_eMoveAbs              := 1,
    eMoverCommand_eMoveModOnTrack       := 2,
    eMoverCommand_eMoveRel              := 3,
    eMoverCommand_eHalt                  := 4,
    eMoverCommand_eGearIn                := 5,
    eMoverCommand_eDynamicsChange        := 6,
    eMoverCommand_eGapChange             := 7,
    eMoverCommand_eSuperImposed          := 8,
    eMoverCommand_eCamInInstant          := 9,
    eMoverCommand_eCamInTwoTables        := 10,
    eMoverCommand_eCamOut                := 11,
    eMoverCommand_eReset                 := 12,
    eMoverCommand_eEnable                := 13,
    eMoverCommand_eDisable               := 14,
    eMoverCommand_eAddMoverToGroup       := 15,
    eMoverCommand_eRemoveMoverFromGroup  := 16,
    eMoverCommand_eMoveAbsDone           := 17,
    eMoverCommand_eMoveModDone           := 18,
```

```
eMoverCommand_eMoveRelDone           := 19,  
eMoverCommand_eHaltCa                 := 20,  
eMoverCommand_eHaltCaDone            := 21,  
eMoverCommand_eMoveAbsQueueing       := 22,  
eMoverCommand_eMoveModQueueing       := 23,  
eMoverCommand_eMoveRelQueueing       := 24,  
eMoverCommand_eGearInInSync          := 25,  
eMoverCommand_eGearOutOfSync         := 26,  
eMoverCommand_eJoinTrack             := 27,  
eMoverCommand_eLeaveTrack             := 28,  
eMoverCommand_eMoveToPosition        := 29,  
eMoverCommand_eMoveZ,  
eMoverCommand_eMoveA,  
eMoverCommand_eMoveB,  
eMoverCommand_eMoveC,  
eMoverCommand_eEnableExternalSetPos,  
eMoverCommand_eDisableExternalSetPos,  
eMoverCommand_eCheckStartConditions,  
eMoverCommand_eAdoptTrackOrientation  
);  
END_TYPE
```

Values

DRAFT | Document in Progress

Name	Description
eMoverCommand_eNoCommand	Idle (no command being processed)
eMoverCommand_eMoveAbs	Absolute movement
eMoverCommand_eMoveModOnTrack	Track movement
eMoverCommand_eMoveRel	Relative (incremental) movement
eMoverCommand_eHalt	Halt breaking ramp
eMoverCommand_eGearIn	Gear in coupling
eMoverCommand_eDynamicsChange	
eMoverCommand_eGapChange	
eMoverCommand_eSuperImposed	
eMoverCommand_eCamInInstant	
eMoverCommand_eCamInTwoTables	
eMoverCommand_eCamOut	
eMoverCommand_eReset	Error reset
eMoverCommand_eEnable	Enabling
eMoverCommand_eDisable	Disabling
eMoverCommand_eAddMoverToGroup	Adding to Collision Avoidance group
eMoverCommand_eRemoveMoverFromGroup	Removing from Collision Avoidance group
eMoverCommand_eMoveAbsDone	
eMoverCommand_eMoveModDone	
eMoverCommand_eMoveRelDone	
eMoverCommand_eHaltCa	
eMoverCommand_eHaltCaDone	
eMoverCommand_eMoveAbsQueueing	
eMoverCommand_eMoveModQueueing	
eMoverCommand_eMoveRelQueueing	
eMoverCommand_eGearInInSync	
eMoverCommand_eGearOutOfSync	
eMoverCommand_eJoinTrack	Joining track
eMoverCommand_eLeaveTrack	Leaving track
eMoverCommand_eMoveToPosition	Free surface movement (XYC coordinates)
eMoverCommand_eMoveZ	Z level movement
eMoverCommand_eMoveA	A angle movement

Name	Description
eMoverCommand_eMoveB	B angle movement
eMoverCommand_eMoveC	C rotation
eMoverCommand_eEnableExternalSetPos	Enabling the external setpoint generator mode
eMoverCommand_eDisableExternalSetPos	Disabling the external setpoint generator mode
eMoverCommand_eCheckStartConditions	Checking enabling conditions

E_PointOption

Circular arc direction specification for track table point. Used for track design.

Syntax

```
TYPE E_PointOption :
(
    none,
    Circle_Clockwise,
    Circle_CounterClockwise
);
END_TYPE
```

Values

Name	Description
none	Must be used for any point other than circular arc
Circle_Clockwise	Clockwise oriented circular arc
Circle_CounterClockwise	Counterclockwise oriented circular arc

E_PointType

Point definition entry in a track table. Used for track design.

Syntax

```
TYPE E_PointType :
(
    none,
    Point,
    Line_Start,
    Line_End,
    Circle_Start,
    Circle_Center,
    Circle_End
);
END_TYPE
```

Values

Name	Description
none	Empty entry
Point	Single point. Track is interpolated through the specified coordinate
Line_Start	Straight line starting point
Line_End	Straight line ending point
Circle_Start	Circular arc starting point
Circle_Center	Circular arc center (radius center)
Circle_End	Circular arc ending point

E_TrackCommandType

Command currently executed (internal handling by the track object).

Syntax

```

TYPE E_TrackCommandType :
(
    eTrackCommand_eNoCommand      := 0,
    eTrackCommand_eEnable         := 1,
    eTrackCommand_eDisable        := 2,
    eTrackCommand_eAppendLine     := 3,
    eTrackCommand_eCloseLoop      := 4,
    eTrackCommand_eStartFromTrack := 5,
    eTrackCommand_eEndAtTrack     := 6,
    eTrackCommand_eAddToGroup     := 7,
    eTrackCommand_eRemoveFromGroup := 8,
    eTrackCommand_eAppendPoint    := 9,
    eTrackCommand_eAppendCircle   := 10,
    eTrackCommand_eBuildTrack     := 11,
    eTrackCommand_eReset          := 12,
    eTrackCommand_eClear          := 13
);
END_TYPE

```

Values

Name	Description
eTrackCommand_eNoCommand	Idle (no command being processed)
eTrackCommand_eEnable	Enabling
eTrackCommand_eDisable	Disabling
eTrackCommand_eAppendLine	
eTrackCommand_eCloseLoop	
eTrackCommand_eStartFromTrack	
eTrackCommand_eEndAtTrack	
eTrackCommand_eAddToGroup	Adding to Collision Avoidance group
eTrackCommand_eRemoveFromGroup	Removing from Collision Avoidance group
eTrackCommand_eAppendPoint	
eTrackCommand_eAppendCircle	
eTrackCommand_eBuildTrack	Building the track geometry
eTrackCommand_eReset	Error reset

eSortingMode

Mover sorting order along tracks in the list.

Syntax

```

TYPE eSortingMode :
(
    ascending    := 0,
    descending   := 1
);
END_TYPE

```

Values

Name	Description
ascending	First to last mover in track's forward direction
descending	First to last mover in track's reverse direction

Structs**ST_ActualMotionParameters**

Current mover situation.

Syntax

```

TYPE ST_ActualMotionParameters :
STRUCT
    Position          : MoverVector;
    fSetTrackPosition : LREAL;
    nTrackOid         : OTCID;

```

```

    sTrackName      : STRING(80);
    fSetVelo        : LREAL;
    fSetAcc         : LREAL;
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
Position	MoverVector	Current position (6D coordinates)
fSetTrackPosition	LREAL	Current position along track (track length in mm)
nTrackOid	OTCID	Track object ID
sTrackName	STRING(80)	Track denomination
fSetVelo	LREAL	Current set velocity
fSetAcc	LREAL	Current set acceleration

ST_BuildTrackParameterLog

Log info for track building operation.

Syntax

```

TYPE ST_BuildTrackParameterLog :
STRUCT
    bSuccess      : BOOL;
    bError        : BOOL;
    nErrorCode    : UDINT;
    fTrackLength  : LREAL;
    sBuildInfoText : STRING;
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
bSuccess	BOOL	Operation successful
bError	BOOL	Error during operation
nErrorCode	UDINT	Error code *
fTrackLength	LREAL	Track length when successfully built
sBuildInfoText	STRING	Info text

* Please refer to the TC3 Motion NC error codes manual or InfoSys for error code description.

ST_CheckStartConditionFeedback

Results of the CheckStartConditions command.

Syntax

```

TYPE ST_CheckStartConditionFeedback :
STRUCT
    nMoverID      : UINT;

```

```

    bPositionOK                : BOOL;
    bPositionXLimitExceeded    : BOOL;
    bPositionYLimitExceeded    : BOOL;
    bAngleLimitExceeded        : BOOL;
    fCurrentAngleDeviation      : LREAL;
    fCurrentPositionDeviationX  : LREAL;
    fCurrentPositionDeviationY  : LREAL;
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
nMoverID	UINT	Mover ID
bPositionOK	BOOL	Current position allows mover enabling
bPositionXLimitExceeded	BOOL	Mover X location too far from C rotation point (quadrant)
bPositionYLimitExceeded	BOOL	Mover Y location too far from C rotation point (quadrant)
bAngleLimitExceeded	BOOL	Current C angle value does not allow X-Y movements
fCurrentAngleDeviation	LREAL	C angle deviation value (in °)
fCurrentPositionDeviationX	LREAL	X distance to closest rotation point (in mm)
fCurrentPositionDeviationY	LREAL	Y distance to closest rotation point (in mm)

ST_CommandReturn

Feedback info returned by any command.

Syntax

```

TYPE ST_CommandReturn :
STRUCT
    Done                : BOOL;
    Busy                : BOOL;
    Aborted              : BOOL;
    Active              : BOOL;
    Error                : BOOL;
    ErrorID              : UDINT;
    stInfo               : ST_CommandReturnInfo;
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
Done	BOOL	Command is done
Busy	BOOL	Command is busy (ordered)
Aborted	BOOL	Command was aborted
Active	BOOL	Command is active (being executed)
Error	BOOL	Error during execution or command refused
ErrorID	UDINT	Error code (can be found in the NC error codes in InfoSys)
stInfo	ST_CommandReturnInfo	Further details relative to an errored command (other object involved)

ST_CommandReturnInfo

Object details involved in an errored command. Mainly used for reporting and identifying collision avoidance results.

Syntax

```

TYPE ST_CommandReturnInfo :
STRUCT
    nObjectType : UDINT;
    nObjectID   : UDINT;
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
nObjectType	UDINT	Object type (Mover, Environment, ...)
nObjectID	UDINT	Object ID. Helps identifying which object is involved in the errored command

ST_DynamicsVADJ

Dynamics constraint values.

Syntax

```

TYPE ST_DynamicsVADJ :
STRUCT
    fVelocity      : LREAL;
    fAcceleration  : LREAL;
    fDeceleration  : LREAL;
    fJerk          : LREAL;
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
fVelocity	LREAL	Velocity value
fAcceleration	LREAL	Acceleration rate
fDeceleration	LREAL	Deceleration rate
fJerk	LREAL	Jerk limitation value

ST_HaltDynamics

Max. dynamics for a mover. Also used by the Halt command.

Syntax

```

TYPE ST_HaltDynamics :
STRUCT
    PathXY      : ST_DynamicsVADJ;
    AxisX       : ST_DynamicsVADJ;
    AxisY       : ST_DynamicsVADJ;
    AxisZ       : ST_DynamicsVADJ;
    AxisA       : ST_DynamicsVADJ;
    AxisB       : ST_DynamicsVADJ;
    AxisC       : ST_DynamicsVADJ;
    Mode        : E_HaltDynamicsMode;
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
PathXY	ST_DynamicsVADJ	Max. values for track movements
AxisX	ST_DynamicsVADJ	Max. values along X direction
AxisY	ST_DynamicsVADJ	Max. values along Y direction
AxisZ	ST_DynamicsVADJ	Max. values along Z direction
AxisA	ST_DynamicsVADJ	Max. values for A rotations
AxisB	ST_DynamicsVADJ	Max. values for B rotations
AxisC	ST_DynamicsVADJ	Max. values for C rotations
Mode	E_HaltDynamicsMode	Defines whether the dynamics set applies to track movements or single coordinate movements

ST_HaltParameter

Halt command limitations.

Syntax

```

TYPE ST_HaltParameter :
STRUCT
    fDeceleration_XY    : LREAL;
    fJerk_XY            : LREAL;
    fDeceleration_A     : LREAL;
    fJerk_A             : LREAL;
    fDeceleration_B     : LREAL;
    fJerk_B             : LREAL;
    fDeceleration_C     : LREAL;
    fJerk_C             : LREAL;
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
fDeceleration_XY	LREAL	Deceleration value for X-Y movement
fJerk_XY	LREAL	Jerk limitation for X-Y movement
fDeceleration_A	LREAL	Deceleration for A rotation
fJerk_A	LREAL	Jerk limitation for A rotation
fDeceleration_B	LREAL	Deceleration for B rotation
fJerk_B	LREAL	Jerk limitation for B rotation
fDeceleration_C	LREAL	Deceleration for C rotation
fJerk_C	LREAL	Jerk limitation for C rotation

ST_JoinTrackLog

Log info for track joining operation.

Syntax

```

TYPE ST_JoinTrackLog :
STRUCT
    nTargetTrack : OTCID;
    fVelo        : LREAL;
    fAcc         : LREAL;
    fJerk        : LREAL;
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
nTargetTrack	OTCID	Track that will be joined
fVelo	LREAL	Velocity value
fAcc	LREAL	Acceleration rate
fJerk	LREAL	Jerk limitation value

ST_LeaveTrackLog

Log info for leaving track operation.

Syntax

```

TYPE ST_LeaveTrackLog :
STRUCT
    fPositionX    : LREAL;
    fPositionY    : LREAL;
    fPositionC    : LREAL;
    fVelo         : LREAL;
    fAcc          : LREAL;
    fDec          : LREAL;
    fJerk         : LREAL;
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
fPositionX	LREAL	Target position X value
fPositionY	LREAL	Target position Y value
fPositionC	LREAL	Target position C angle
fVelo	LREAL	Velocity value
fAcc	LREAL	Acceleration rate
fDec	LREAL	Deceleration rate
fJerk	LREAL	Jerk limitation value

ST_List

Mover info used by the sorting list function block.

Syntax

```

TYPE ST_List :
STRUCT
    nNumInList    : INT;
    aMoverList    : ARRAY[1..maxNumMover] OF I_XPlanarMoverStandard;
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
nNumInList	INT	Index in the list
aMoverList	ARRAY[1..max NumMover] OF I_XPlanarMover Standard	Mover array handled by the list sorting function block

ST_LoggingVisu

Visualization data for the logging page.

Syntax

```

TYPE ST_LoggingVisu :
STRUCT
    nScrollBarMin      : UINT;
    nScrollBarMax      : UINT;
    nScrollBarValue    : UINT;
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
nScrollBarMin	UINT	Scroll bar minimum edge
nScrollBarMax	UINT	Scroll bar maximum edge
nScrollBarValue	UINT	Scroll bar scrolling position

ST_MoveOnTrackParametersLog

Log info for track movement.

Syntax

```

TYPE ST_MoveOnTrackParametersLog:
STRUCT
    fTargetPosition    : LREAL;
    fVelo              : LREAL;
    nTargetTrackOid     : OTCID;
    sTargetTrack        : STRING(80);
    nTargetTrackID      : UDINT;
    fAcc                : LREAL;
    fDec                : LREAL;
    fJerk               : LREAL;
    fGap                : LREAL;
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
fTargetPosition	LREAL	Target position along track
fVelo	LREAL	Velocity value
nTargetTrackOid	OTCID	Target track object ID
sTargetTrack	STRING(80)	Target track denomination
nTargetTrackID	UDINT	Target track index (ID from 1 to n)
fAcc	LREAL	Acceleration rate
fDec	LREAL	Deceleration rate
fJerk	LREAL	Jerk limitation value
fGap	LREAL	Gap to predecessor mover kept when queued-up

ST_MoverCommand

Mover commands combination used for internal command handling.

Syntax

```

TYPE ST_MoverCommand : //structure for storing a list of commands to the mover
STRUCT
    sStation           : STRING;    // Station Name that give a move command
    eCommandType       : E_MoverCommandType; // what command was done
    stActualParameter  : ST_ActualMotionParameters;
    stMoveOnTrackParameter : ST_MoveOnTrackParametersLog;
    stMoveToPositionXYC : ST_MoveToPositionXYCParametersLog;
    stHaltParameter    : ST_HaltParameter;
    stJoinTrackParameter : ST_JoinTrackLog;
    stLeaveTrackParameter : ST_LeaveTrackLog;
    sText              : STRING(255);
    stTimeStamp        : STRING ; //dcTimestampTime can be decoded later
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
sStation	STRING	Station name that instructed the command
eCommandType	E_MoverCommandType	Command type
stActualParameter	ST_ActualMotionParameters	Actual motion parameters
stMoveOnTrackParameter	ST_MoveOnTrackParametersLog	Log info for track movement commands
stMoveToPositionXYC	ST_MoveToPositionXYCParametersLog	Log info for free movement commands
stHaltParameter	ST_HaltParameter	Max. dynamics used by Halt command
stJoinTrackParameter	ST_JoinTrackLog	Log info for track joining operation
stLeaveTrackParameter	ST_LeaveTrackLog	Log info for track leaving operation
sText	STRING(255)	Description text
stTimeStamp	STRING	Time stamp

ST_MoverCommandLog

Log info for any mover command.

Syntax

```

TYPE ST_MoverCommandLog :
STRUCT
    nNo          : INT;
    sTimeStamp    : STRING;
    sTimeStampDate : STRING;
    sTimeStampTime : STRING;
    sStation      : STRING;
    sCommandType  : STRING;
    sText         : STRING(255);
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
nNo	INT	Index in log table
sTimeStamp	STRING	Complete time stamp
sTimeStampDate	STRING	Time stamp date part
sTimeStampTime	STRING	Time stamp time part
sStation	STRING	Station name that instructed the command
sCommandType	STRING	Command type
sText	STRING(255)	Description text

ST_MoverInList

Mover info used by sorting list.

Syntax

```

TYPE ST_MoverInList :
STRUCT
    nMoverIndex      : UDINT;
    nMoverPosition   : LREAL;
    nMoverTrackOID   : UDINT;
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
nMoverIndex	UDINT	Mover index (1 to n)
nMoverPosition	LREAL	Mover position on track
nMoverTrackOID	UDINT	Track object ID of the track the mover is currently located on

ST_MoverVisualisation

Visualization info for a mover object.

Syntax

```

TYPE ST_MoverVisualisation :
STRUCT
    stMoverPos      : MoverVector;
    stMoverVelo     : MoverVector;
    stMoverAcc      : MoverVector;
    nID             : UDINT;
    trackPos        : LREAL;
    bError          : BOOL;
    nErrorID        : UDINT;
    eState          : MC_PLANAR_STATE;
    eActivationState : MC_PLANAR_MOVER_COMMAND_MODE;
    bBusy           : BOOL;
    bActivated      : BOOL;

```

```

    bOnTrack          : BOOL;
    bActivating       : BOOL;
    bInactive         : BOOL;
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
stMoverPos	MoverVector	Mover actual position
stMoverVelo	MoverVector	Mover actual velocity
stMoverAcc	MoverVector	Mover actual acceleration
nID	UDINT	Mover ID
trackPos	LREAL	Position on track
bError	BOOL	Mover error
nErrorID	UDINT	Mover error ID
eState	MC_PLANAR_STATE	Mover state
eActivationState	MC_PLANAR_MOVER_COMMAND_MODE	Mover command mode
bBusy	BOOL	Mover is busy
bActivated	BOOL	Mover is active
bOnTrack	BOOL	Mover is on track
bActivating	BOOL	Mover being activated
bInactive	BOOL	Mover is inactive

ST_MoveToPositionXYCParametersLog

Log info for move to position operation.

Syntax

```

TYPE ST_MoveToPositionXYCParametersLog:
STRUCT
    fVelo          : LREAL;
    fTargetPositionX : LREAL;
    fTargetPositionY : LREAL;
    fTargetPositionC : LREAL;
    fAcc           : LREAL;
    fDec           : LREAL;
    fJerk          : LREAL;
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
fVelo	LREAL	Velocity value
fTargetPositionX	LREAL	Target position X value
fTargetPositionY	LREAL	Target position Y value
fTargetPositionC	LREAL	Target position C angle
fAcc	LREAL	Acceleration rate
fDec	LREAL	Deceleration rate
fJerk	LREAL	Jerk limitation value

ST_StartEndAtTrackParameterLog

Log info relative to track network predecessor or successor.

Syntax

```

TYPE ST_StartEndAtTrackParameterLog :
STRUCT
    nTrackOID      : UDINT;
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
nTrackOID	UDINT	Track object ID

ST_TrackAppendCircleParameterLog

Log info for circular arc segment appending operation.

Syntax

```

TYPE ST_TrackAppendCircleParameterLog:
STRUCT
    startPositionX      : LREAL;
    startPositionY      : LREAL;
    centerPositionX      : LREAL;
    centerPositionY      : LREAL;
    endPositionX          : LREAL;
    endPositionY          : LREAL;
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
startPositionX	LREAL	Starting point X value
startPositionY	LREAL	Starting point Y value
centerPositionX	LREAL	Arc radius center X value
centerPositionY	LREAL	Arc radius center Y value
endPositionX	LREAL	End point X value
endPositionY	LREAL	End point Y value

ST_TrackAppendLineParameterLog

Log info for straight line segment appending operation.

Syntax

```

TYPE ST_TrackAppendLineParameterLog :
STRUCT
    startPositionX      : LREAL;
    startPositionY      : LREAL;
    endPositionX        : LREAL;
    endPositionY        : LREAL;
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
startPositionX	LREAL	Starting point X value
startPositionY	LREAL	Starting point Y value
endPositionX	LREAL	End point X value
endPositionY	LREAL	End point Y value

ST_TrackAppendPointParameterLog

Log info for single point appending operation.

Syntax

```

TYPE ST_TrackAppendPointParameterLog :
STRUCT
    positionX          : LREAL;
    positionY          : LREAL;
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
positionX	LREAL	Point X value
positionY	LREAL	Point Y value

ST_TrackBuildParameter

Additional track parameter used to build it.

Syntax

```

TYPE ST_TrackBuildParameter :
STRUCT
    StartFromTrack      : I_TrackStandard;
    EndAtTrack          : I_TrackStandard;
    bCloseTheLoop       : BOOL;
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
StartFromTrack	I_TrackStandard	Track network : predecessor track
EndAtTrack	I_TrackStandard	Track network : successor track
bCloseTheLoop	BOOL	Closed loop track. Allows modulo movement

ST_TrackBuiltInfo

Track building feedback data.

Syntax

```

TYPE ST_TrackBuiltInfo :
STRUCT
    nTrackBuiltCountInfo : UDINT;
    TrackInfo             : ARRAY[1..maxTrackCount] OF ST_TrackInfo;
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
nTrackBuiltCountInfo	UDINT	Quantity of tracks built
TrackInfo	ARRAY[1..maxTrackCount] OF ST_TrackInfo	Track relationship info for each track

ST_TrackCommand

Track commands combination used for internal command handling.

Syntax

```

TYPE ST_TrackCommand :
STRUCT
    eCommandType          : E_TrackCommandType; // what command was done
    stTrackAppendLine      : ST_TrackAppendLineParameterLog;
    stTrackAppendPoint     : ST_TrackAppendPointParameterLog;
    stTrackAppendCircle    : ST_TrackAppendCircleParameterLog;
    stBuildTrack           : ST_BuildTrackParameterLog;
    stEndAtTrack           : ST_StartEndAtTrackParameterLog;
    stStartFromTrack       : ST_StartEndAtTrackParameterLog;
    sText                  : STRING(255);
    stTimeStamp            : STRING; //dcTimestampTime can be decoded later
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
eCommandType	E_TrackCommandType	Command type
stTrackAppendLine	ST_TrackAppendLineParameterLog	Log info for straight line appending command
stTrackAppendPoint	ST_TrackAppendPointParameterLog	Log info for single point appending command
stTrackAppendCircle	ST_TrackAppendCircleParameterLog	Log info for circular arc appending command
stBuildTrack	ST_BuildTrackParameterLog	Log info for track building command
stEndAtTrack	ST_StartEndAtTrackParameterLog	Log info for successor track
stStartFromTrack	ST_StartEndAtTrackParameterLog	Log info for predecessor track
sText	STRING(255)	Description text
stTimeStamp	STRING	Time stamp

ST_TrackCommandLog

Log info for track commands.

Syntax

```

TYPE ST_TrackCommandLog :
STRUCT
    nNo          : INT;
    stTimeStamp   : STRING;
    stTimeStampDate : STRING;
    stTimeStampTime : STRING;

```

```

        sCommandType      : STRING;
        sText              : STRING(255);
    END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
nNo	INT	Index
sTimeStamp	STRING	Time stamp
sTimeStampDate	STRING	Time stamp date part
sTimeStampTime	STRING	Time stamp time part
sCommandType	STRING	Command type
sText	STRING(255)	Description

ST_TrackInfo

Track relationship info.

Syntax

```

TYPE ST_TrackInfo :
STRUCT
    TrackOID          : UDINT;
    StartFromTrack    : I_TrackStandard;
    EndAtTrack        : I_TrackStandard;
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
TrackOID	UDINT	Track object ID
StartFromTrack	I_TrackStandard	Predecessor track
EndAtTrack	I_TrackStandard	Successor track

ST_TrackTable

Track geometry segment. Used to define a track segment.

Syntax

```

TYPE ST_TrackTable :
STRUCT
    fPointPosition_X   : LREAL;
    fPointPosition_Y   : LREAL;
    fPointOrientation_C : LREAL;
    ePointType          : E_PointType;
    eOption             : E_PointOption;
END_STRUCT
END_TYPE

```

Parameters

Name	Type	Description
fPointPosition_X	LREAL	Point X value
fPointPosition_Y	LREAL	Point Y value
fPointOrientation_C	LREAL	Point C value
ePointType	E_PointType	Point type (segment part definition)
eOption	E_PointOption	Point definition options, depending on point type

Functions**ConvertToEventEntry**

Converts the event info into an event logger entry data type.

Return value

TcEventEntry

Inputs

Name	Type	Description
nEventId	UDINT	Event ID
uuidEventClass	GUID	Event object GUID

MoverVectorToPositionXYC

Converts a 6D position vector to a PositionXYC data type, required for some movement commands.

Return value

REFERENCE TO PositionXYC

Inputs

Name	Type	Description
stPosMoverVector	MoverVector	6D position vector

Function Blocks

FB_List

Sortable list which allows generation of mover data queues in correct order, based on their location on tracks.

Do not call the main FB directly. Only use the available methods and properties.

Methods

Name	Description
AddMover	Adds a mover to the list
AddTrack	Adds a track to the list
ClearList	Clears all list contents
ClearTrackOrder	Clears the track order (track network)
GetMoverInListPosition	Gets mover info by index
RemoveFirstInList	Extracts the first mover info from the list
RemoveFromList	Removes a mover from the list
SortList	Sorts the list content

AddMover

Adds a mover object to the list. A return value = 1 means successful execution. A return value < 0 means an error occurred. Please refer to the object's command history for execution results description.

Syntax

```
METHOD AddMover : DINT
VAR_INPUT
    ipMover      : I_XPlanarMoverStandard;
END_VAR
```

Return value

DINT

Inputs

Name	Type	Description
ipMover	I_XPlanarMoverStandard	Mover object to be added to the list

AddTrack

Adds a track object to the list.

Syntax

```
METHOD AddTrack : DINT
VAR_INPUT
    i_Track      : I_TrackStandard;
END_VAR
```

Return value

DINT

Inputs

Name	Type	Description
i_Track	I_TrackStandard	Track object to be added to the list

ClearList

Clears all list content (movers and tracks).

Syntax

```
METHOD ClearList : BOOL
```

Return value

BOOL

ClearTrackOrder

Clears the tracks from the list.

Syntax

```
METHOD ClearTrackOrder : BOOL
```

Return value

BOOL

GetMoverInListPosition

Returns the mover object from the list for the specified index. The mover object is not deleted from the list.

Syntax

```
METHOD GetMoverInListPosition : I_XPlanarMoverStandard
VAR_INPUT
    nListPosition      : UDINT;
END_VAR
```

Return value

I_XPlanarMoverStandard

Inputs

Name	Type	Description
nListPosition	UDINT	List index (position)

RemoveFirstInList

Returns the first mover of the list (after sorting). Deletes the mover from the list.

Syntax

```
METHOD RemoveFirstInList : I_XPlanarMoverStandard
```

Return value

I_XPlanarMoverStandard

RemoveFromList

Removes a mover object from the list. A return value = 1 means successful execution. A return value < 0 means an error occurred. Please refer to the object's command history for execution results description.

Syntax

```
METHOD RemoveFromList : DINT
VAR_INPUT
    ipMover      : I_XPlanarMoverStandard;
END_VAR
```

Return value

DINT

Inputs

Name	Type	Description
ipMover	I_XPlanarMoverStandard	Mover object to be removed from the list

SortList

Sorts the movers in the list based on their location on the tracks set in the list. Sorting mode (ascending or descending) determines whether the tracks (track network) are seen from the end or from the start.

Syntax

```
METHOD SortList : INT
```

Return value

INT

 Properties

Name	Type	Access	Description
P_FirstInList	I_XPlanarMoverStandard	RO	First mover in the list (queue). Mover gets deleted from the list when returned
P_Message	STRING	RO	Result description as message
P_MoverList	REFERENCE TO ARRAY [1..maxNumMover] OF ST_MoverInList	RO	Complete list of movers contained in the FB
P_NumberInList	UDINT	RO	List count. Returns the quantity of movers in the list
P_Result	DINT	RO	Return value for adding or removing an object to or from the list
P_SortingMode	eSortingMode	RW	List sorting mode. Determines whether the tracks are searched from the end or from the start position
P_SortingOrigin	LREAL	RW	Point of origin for the sorting method
P_TrackBuiltInfo	ST_TrackBuiltInfo	RW	Track layout relationships info, required by the sorting process to order the movers in the list

FB_TrackStandard

Track object used to operate movers on a one-dimensional path, freely designed on the XPlanar stator surface.

Do not call the main FB directly. Only use the available methods and properties.

Methods

Name	Description
AddToGroup	Adds the track to the Collision Avoidance group
AppendCircle	Appends a circular arc segment to the track's geometry
AppendLine	Appends a straight-line segment to the track's geometry
AppendPoint	Appends a single point segment to the track's geometry
BuildTrackExt	Builds the track according to the defined geometry
Clear	Clears the track geometry
CloseLoop	Defines the track as closed loop (gap between the end point and the starting point is linked by an interpolated segment)
Cycle	Cyclic operations of FB_TrackStandard
Disable	Disables the track
Enable	Enables the track
EndAtTrack	Sets the track's successor in a track network
GetCommandLog	Returns the log info at a specified index
GetLength	Returns the 1D track length when it was built
GetMoverPositionAt	Returns the X-Y coordinates of a point along the track
GetTrackPositionAt	Returns the closest position on track for a X-Y coordinate
RemoveFromGroup	Removes the track from the Collision Avoidance group
Reset	Resets any error returned by the track object
StartFromTrack	Sets the track's predecessor in a track network
TrackCommandToString	Returns the entered command in string format
UpdateCommandHistory	Adds an entry in the log history

AddToGroup

Adds the track to the Collision Avoidance group. The track must be enabled before it can be added to the group.

Syntax

```
METHOD AddToGroup : BOOL
VAR_INPUT
    group : REFERENCE TO MC_PlanarGroup;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
group	REFERENCE TO MC_PlanarGroup	Collision Avoidance group for the current application

AppendCircle

Appends a circular arc segment to the track geometry.

Syntax

```
METHOD AppendCircle : BOOL
VAR_INPUT
    startPosition : REFERENCE TO PositionXYC;
    centerPosition : REFERENCE TO PositionXY;
    endPosition : REFERENCE TO PositionXYC;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
startPosition	REFERENCE TO PositionXYC	Starting point coordinates
centerPosition	REFERENCE TO PositionXY	Arc center point coordinates
endPosition	REFERENCE TO PositionXYC	End point coordinates

AppendLine

Appends a straight-line segment to the track geometry.

Syntax

```
METHOD AppendLine : BOOL
VAR_INPUT
    startPosition      : REFERENCE TO PositionXYC;
    endPosition        : REFERENCE TO PositionXYC;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
startPosition	REFERENCE TO PositionXYC	Starting point coordinates
endPosition	REFERENCE TO PositionXYC	End point coordinates

AppendPoint

Appends a single point (passing point) to the track geometry. Used to define a specific passing point for an interpolated track segment.

Syntax

```
METHOD AppendPoint : BOOL
VAR_INPUT
    position          : REFERENCE TO PositionXYC;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
position	REFERENCE TO PositionXYC	Passing point coordinates

BuildTrackExt

Builds the track object according to its defined geometry.

Syntax

```
METHOD BuildTrackExt : BOOL
VAR_INPUT
    TrackTable    : REFERENCE TO FB_TrackTable;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
TrackTable	REFERENCE TO FB_TrackTable	Track geometry used to build the track

Clear

Clears the existing track geometry.

Syntax

```
METHOD Clear : BOOL
```

Return value

BOOL

CloseLoop

Sets the track as a closed loop. Allows modulo movement operations. No predecessor and/or successor must be set for a closed loop track.

Syntax

```
METHOD CloseLoop : BOOL
```

Return value

BOOL

Cycle

Cyclic update operations. Handles communication with the process image. Method must be called every PLC cycle.

Syntax

```
METHOD Cycle : BOOL
```

Return value

BOOL

Disable

Disables the track. Movements on the track are prohibited. Once disabled, the track gets automatically removed from the Collision Avoidance group.

Syntax

```
METHOD Disable : BOOL
```

Return value

BOOL

Enable

Enables the track. Movements on the track are allowed. After enabling, the track is not automatically included in the Collision Avoidance monitoring.

Syntax

```
METHOD Enable : BOOL
```

Return value

BOOL

EndAtTrack

Sets the track's successor in a track network.

Syntax

```
METHOD EndAtTrack : BOOL
VAR_INPUT
    ipTrackExt : I_TrackStandard;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
ipTrackExt	I_TrackStandard	Successor track object

GetCommandLog

Returns an entry from the log info at the specified index.

Syntax

```
METHOD GetCommandLog : ST_TrackCommandLog
VAR_INPUT
    nNum : UINT;
END_VAR
```

Return value

ST_TrackCommandLog

Inputs

Name	Type	Description
nNum	UINT	Index of the log info to be returned from the history

GetLength

Gets the track length (one-dimensional path length) in mm.

Syntax

```
METHOD GetLength : LREAL
```

Return value

LREAL

GetMoverPositionAt

Returns the coordinates of the specified position along the track. If the specified position is out of track limits, the returned coordinates will be 0, 0, 0 (PositionXYC).

Syntax

```
METHOD GetMoverPositionAt : BOOL
VAR_INPUT
    (* Arc length value where the position is evaluated. *)
    arcLength : LREAL;
END_VAR
VAR_IN_OUT
    (* The position at the specified arc parameter. *)
    position : PositionXYC;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
arcLength	LREAL	Position along the track

In/Outputs

Name	Type	Description
position	PositionXYC	Resulting coordinates returned by the method

GetTrackPositionAt

Returns the closest position along the track of the specified coordinates.

Syntax

```
METHOD GetTrackPositionAt : LREAL
VAR_INPUT
    Position      : REFERENCE TO PositionXYC;
END_VAR
```

Return value

LREAL

Inputs

Name	Type	Description
Position	REFERENCE TO PositionXYC	Coordinates to be checked for

RemoveFromGroup

Removes the track from the Collision Avoidance group. Once disabled, the track gets automatically removed from the Collision Avoidance group.

Syntax

```
METHOD RemoveFromGroup : BOOL
```

Return value

BOOL

Reset

Resets any pending error on the track object whenever the track's state is set to Error.

Syntax

```
METHOD Reset : BOOL
```

Return value

BOOL

StartFromTrack

Sets the track's predecessor in a track network. The predecessor's geometry must already exist (predecessor track must be built).

Syntax

```
METHOD StartFromTrack : BOOL
VAR_INPUT
    ipTrackExt : I_TrackStandard;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
ipTrackExt	I_TrackStandard	Predecessor track object

TrackCommandToString

Converts a track command to text for logging.

Syntax

```
METHOD TrackCommandToString : STRING
VAR_INPUT
    eTrackCommand : E_TrackCommandType;
END_VAR
```

Return value

STRING

Inputs

Name	Type	Description
eTrackCommand	E_TrackCommandType	Command object

UpdateCommandHistory

Appends a log info entry into the log history.

Syntax

```
METHOD UpdateCommandHistory
VAR_IN_OUT
    stCommand : ST_TrackCommand;
END_VAR
```

Inputs

Name	Type	Description
stCommand	ST_TrackCommand	Command object info

 Properties

Name	Type	Access	Description
P_AddToGroup	REFERENCE TO ST_CommandReturn	RO	Command feedback for AddToGroup
P_AppendCircle	REFERENCE TO ST_CommandReturn	RO	Command feedback for AppendCircle
P_AppendLine	REFERENCE TO ST_CommandReturn	RO	Command feedback for AppendLine
P_AppendPoint	REFERENCE TO ST_CommandReturn	RO	Command feedback for AppendPoint
P_BuildTrackExt	REFERENCE TO ST_CommandReturn	RO	Command feedback for BuildTrackExt
P_BuildTrackExtInfoText	STRING	RO	Error description of the BuildTrackExt command
P_CloseLoop	REFERENCE TO ST_CommandReturn	RO	Command feedback for CloseLoop
P_Disable	REFERENCE TO ST_CommandReturn	RO	Command feedback for Disable
P_Enable	REFERENCE TO ST_CommandReturn	RO	Command feedback for Enable
P_EndAtTrack	REFERENCE TO ST_CommandReturn	RO	Command feedback for EndAtTrack
P_GroupOID	UDINT	RO	Collision Avoidance group object ID. 0 means track is not part of the group
P_Info	PlanarObjectInfo	RO	Returns the object info of the track
P_MoverCountOnTrack	UDINT	RO	Returns the current amount of movers present on the track
P_OID	UDINT	RO	Returns the track's object ID
P_OperationMode	MC_PLANAR_TRACK_OPERATION_MODE	RO	Returns the track's current operation mode
P_RemoveFromGroup	REFERENCE TO ST_CommandReturn	RO	Command feedback for RemoveFromGroup
P_Reset	REFERENCE TO ST_CommandReturn	RO	Command feedback for Reset
P_StartFromTrack	REFERENCE TO ST_CommandReturn	RO	Command feedback for StartFromTrack
P_State	MC_PLANAR_STATE	RO	Returns the track's current state
P_TrackBuiltInfo	ST_TrackBuiltInfo	RO	Returns the track geometry info used to build the track

FB_TrackTable

Track geometry data container. Used to define the track segments for building a track.

Do not call the main FB directly. Only use the available methods and properties.

Methods

Name	Description
AddPoint	Adds a point (or segment) to the geometry. Order of segments is important and must be considered
CheckTrackTable	Checks the geometry for plausibility. Attempts to avoid track building failures
ClearTrackTable	Clears the geometry content

AddPoint

Adds a track point (or segment) to the track's geometry. The order in which segments are added is important and must be considered.

Syntax

```
METHOD AddPoint : INT
VAR_INPUT
    ePointType      : E_PointType;
    eOption          : E_PointOption;
    Position_X       : LREAL;
    Position_Y       : LREAL;
    Orientation_C     : LREAL;
END_VAR
```

Return value

INT

Inputs

Name	Type	Description
ePointType	E_PointType	Point type definition
eOption	E_PointOption	Additional options, depending on the point type
Position_X	LREAL	Point coordinate X
Position_Y	LREAL	Point coordinate Y
Orientation_C	LREAL	Point orientation C

CheckTrackTable

Checks the geometry definition for plausibility.

Syntax

```
METHOD CheckTrackTable : DINT
```

Return value

DINT

ClearTrackTable

Clears the track geometry content.

Syntax

```
METHOD ClearTrackTable : BOOL
```

Return value

BOOL

 Properties

Name	Type	Access	Description
P_BuildParameter	ST_TrackBuildParameter	RO	Returns the track network relationship info
P_CloseTheLoop	BOOL	RW	Track geometry is a closed loop
P_EndAtTrack	I_TrackStandard	RW	Successor track in a track network
P_InfoText	STRING	RO	Description text
P_NumberOfPointsInTable	UINT	RO	Quantity of segments in the geometry container
P_StartFromTrack	I_TrackStandard	RW	Predecessor track in a track network
P_Table	REFERENCE TO ARRAY[1..maxNumTrackPoints] OF ST_TrackTable	RO	Returns the complete track geometry content
P_TrackCheckImplausibleTableIndex	UINT	RO	Geometry segment index after which the plausibility check failed
P_TrackTableCheckResult	BOOL	RO	Results of the geometry plausibility test

FB_XPlanarMoverStandard

Mover object used to operate movements with an XPlanar system.

Do not call the main FB directly. Only use the available methods and properties.

Methods

Name	Description
AdoptTrackOrientation	Turns the C axis according to the defined C coordinate value of the track on which the mover is standing
CheckStartConditions	Checks if the mover is allowed to move from its current position
Cycle	Cyclic operations of FB_XPlanarMoverStandard
Disable	Disables the mover
DisableExternalSetpointGeneration	Stops the external setpoint generator mode
Enable	Enables the mover
EnableExternalSetpointGeneration	Starts the external setpoint generator mode
GetCommandLog	Gets the log info entry at a specified index
Halt	Halts the mover
JoinTrack	Joins the specified track
LeaveTrack	Leaves the current track
MoveA	Tilts the mover by its A axis (around X axis)
MoveB	Tilts the mover by its B axis (around Y axis)
MoveC	Rotates the mover by its C axis (around Z axis)
MoveOnTrack	Moves along a track or a track network
MoverCommandToString	Converts a command to text for logging
MoveToPosition	Moves freely over the surface (XYC coordinates movement)
MoveZ	Raises or lowers the mover's altitude (flight height)
Reset	Resets the mover if errored
SetExternalSetpointValues	Transmits the external setpoint values to the mover (during external setpoint generator mode)
UpdateCommandHistory	Adds a log entry in the history

AdoptTrackOrientation

Aligns the C axis to the defined angle of the track the mover is currently standing on. C rotation is a modulo operation. The C rotation can only be performed at the intersection of 4 quadrants, which corresponds to every 120 mm in X and Y directions over the stator surface. The property P_AtRotationPoint returns whether the mover is allowed to perform a C rotation, based on its current location.

Syntax

```
METHOD AdoptTrackOrientation : BOOL
VAR_INPUT
    slogMessage : STRING;
    velocity     : LREAL;
    acceleration : LREAL;
    deceleration : LREAL;
    jerk        : LREAL;
    stOptions    : REFERENCE TO ST_AdoptTrackOrientationOptions;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
slogMessage	STRING	Log info for the command
velocity	LREAL	Velocity value
acceleration	LREAL	Acceleration rate
deceleration	LREAL	Deceleration rate
jerk	LREAL	Jerk limitation value
stOptions	REFERENCE TO ST_AdoptTrack OrientationOptions	Additional command parameters (related to modulo movement)

CheckStartConditions

Checks whether the mover is allowed to move from its current position, based on its current C angle value.

Syntax

```
METHOD CheckStartConditions : ST_CheckStartConditionFeedback
```

Return value

ST_CheckStartConditionFeedback

Cycle

Cyclic update operations. Handles communication with the process image. Method must be called every PLC cycle.

Syntax

```
METHOD Cycle : BOOL
```

Return value

BOOL

Disable

Disables the mover. Once disabled, the mover gets automatically removed from the Collision Avoidance group and eventually the track it was standing on. A disabled mover is never considered on a track.

Syntax

```
METHOD Disable : BOOL
VAR_INPUT
    sLogMessage : STRING;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
sLogMessage	STRING	Log info for the command

DisableExternalSetpointGeneration

Disables the external setpoint generator mode. The mover returns to its previous command mode.

Syntax

```
METHOD DisableExternalSetpointGeneration : BOOL
VAR_INPUT
    slogMessage : STRING;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
slogMessage	STRING	Log info for the command

Enable

Enables the mover. After enabling, the mover is not automatically included in the Collision Avoidance monitoring.

Syntax

```
METHOD Enable : BOOL
VAR_INPUT
    slogMessage : STRING;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
slogMessage	STRING	Log info for the command

EnableExternalSetpointGeneration

Enables the external setpoint generator mode. Cyclic setpoint values must be supplied until this mode is disabled again. Collision Avoidance monitoring cannot interfere in this mode.

Syntax

```
METHOD EnableExternalSetpointGeneration : BOOL
VAR_INPUT
    slogMessage      : STRING;
    eSetpointMode    : MC_EXTERNAL_SET_POSITION_MODE;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
slogMessage	STRING	Log info for the command
eSetpointMode	MC_EXTERNAL_SET_POSITION_MODE	External setpoint mode (absolute or relative). Influences whether or not the mover switches to the External Setpoint Generator command mode

GetCommandLog

Returns an entry from the log info at the specified index.

Syntax

```
METHOD GetCommandLog : ST_MoverCommandLog
VAR_INPUT
    nNum : UINT;
END_VAR
```

Return value

ST_MoverCommandLog

Inputs

Name	Type	Description
nNum	UINT	Index of the log info to be returned from the history

Halt

Stops the mover with the predefined max. dynamics. The current Command Mode has no influence on the halt dynamics.

Syntax

```
METHOD Halt : BOOL
VAR_INPUT
    slogMessage : STRING;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
slogMessage	STRING	Log info for the command

JoinTrack

The mover joins the specified track with the specified dynamics, if possible. If the track is unreachable, the error details can be read from the command feedback property. To execute this command, both the mover and the track must be enabled.

Syntax

```
METHOD JoinTrack : BOOL
VAR_INPUT
    slogMessage      : STRING;
    ipTrack           : I_TrackStandard;
    fVelo             : LREAL;
    fAcceleration     : LREAL;
    fJerk             : LREAL;
    stOptions         : REFERENCE TO ST_JoinTrackOptions;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
slogMessage	STRING	Log info for the command
ipTrack	l_TrackStandard	Target track to be joined
fVelo	LREAL	Velocity value to reach the track
fAcceleration	LREAL	Acceleration rate for the joining movement
fJerk	LREAL	Jerk limitation value for the joining movement
stOptions	REFERENCE TO ST_JoinTrackOptions	Additional command parameters

LeaveTrack

The mover leaves the track it is standing on and moves to the specified target position in FreeMovement mode with the specified dynamics. To execute this command, both the mover and the track must be enabled. A disabled mover is never considered on a track.

Syntax

```

METHOD LeaveTrack : BOOL
VAR_INPUT
    slogMessage : STRING;
    position     : REFERENCE TO PositionXYC;
    velocity     : LREAL;
    acceleration : LREAL;
    deceleration : LREAL;
    jerk        : LREAL;
    stOptions    : REFERENCE TO ST_LeaveTrackOptions;
END_VAR

```

Return value

BOOL

Inputs

Name	Type	Description
slogMessage	STRING	Log info for the command
position	REFERENCE TO PositionXYC	Target position
velocity	LREAL	Velocity value
acceleration	LREAL	Acceleration rate
deceleration	LREAL	Deceleration rate
jerk	LREAL	Jerk limitation value
stOptions	REFERENCE TO ST_LeaveTrack Options	Additional command parameters

MoveA

Tilts the mover around its X axis.

Syntax

```

METHOD MoveA : BOOL
VAR_INPUT
    slogMessage      : STRING;
    targetPosition    : LREAL;
    velocity          : LREAL;
    acceleration       : LREAL;
    deceleration       : LREAL;
    jerk              : LREAL;
END_VAR

```

Return value

BOOL

Inputs

Name	Type	Description
slogMessage	STRING	Log info for the command
targetPosition	LREAL	Target A angle
velocity	LREAL	Velocity value
acceleration	LREAL	Acceleration rate
deceleration	LREAL	Deceleration rate
jerk	LREAL	Jerk limitation value

MoveB

Tilts the mover around its Y axis.

Syntax

```
METHOD MoveB : BOOL
VAR_INPUT
    slogMessage      : STRING;
    targetPosition    : LREAL;
    velocity          : LREAL;
    acceleration      : LREAL;
    deceleration      : LREAL;
    jerk              : LREAL;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
slogMessage	STRING	Log info for the command
targetPosition	LREAL	Target B angle
velocity	LREAL	Velocity value
acceleration	LREAL	Acceleration rate
deceleration	LREAL	Deceleration rate
jerk	LREAL	Jerk limitation value

MoveC

Rotates the mover around its Z axis. C rotation is a modulo operation. The C rotation can only be performed at the intersection of 4 quadrants, which corresponds to every 120 mm in X and Y directions over the stator surface. The property P_AtRotationPoint returns whether the mover is allowed to perform a C rotation, based on its current location.

Syntax

```
METHOD MoveC : BOOL
VAR_INPUT
    slogMessage      : STRING;
    targetPosition    : LREAL;
    velocity          : LREAL;
    acceleration      : LREAL;
    deceleration      : LREAL;
    jerk              : LREAL;
    stOptions         : REFERENCE TO ST_MoveCOptions;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
slogMessage	STRING	Log info for the command
targetPosition	LREAL	Target C angle
velocity	LREAL	Velocity value
acceleration	LREAL	Acceleration rate
deceleration	LREAL	Deceleration rate
jerk	LREAL	Jerk limitation value
stOptions	REFERENCE TO ST_MoveCOpti ons	Additional command parameters (related to modulo movement)

MoveOnTrack

Moves the mover along a track or a track network. The mover follows the one-dimensional path defined by the track or the track network.

Syntax

```

METHOD MoveOnTrack : BOOL
VAR_INPUT
    slogMessage      : STRING;
    ipTrack          : I_TrackStandard;
    trackPosition    : LREAL;
    velocity         : LREAL;
    acceleration     : LREAL;
    jerk            : LREAL;
    gap             : LREAL;
    direction        : Tc3_Mc3PlanarMotion.MC_DIRECTION :=
Tc3_Mc3PlanarMotion.MC_DIRECTION.mcDirectionPositive;
END_VAR

```

Return value

BOOL

Inputs

Name	Type	Description
sLogMessage	STRING	Log info for the command
ipTrack	I_TrackStandard	Target track object. Can be different than the current track if it is part of a track network
trackPosition	LREAL	Target position on the target track
velocity	LREAL	Velocity value
acceleration	LREAL	Acceleration rate
jerk	LREAL	Jerk limitation value
gap	LREAL	Gap to keep with predecessor mover in case of queueing (if target position cannot be reached)
direction	Tc3_Mc3PlanarMotion.MC_DIRECTION	Movement direction (modulo movement)

MoverCommandToString

Converts a command to text for logging.

Syntax

```
METHOD MoverCommandToString : STRING
VAR_INPUT
    eMoverCommand      : E_MoverCommandType;
END_VAR
```

Return value

STRING

Inputs

Name	Type	Description
eMoverCommand	E_MoverCommandType	Mover command

MoveToPosition

The mover moves in a straight line over the stator surface to the specified target position with the specified dynamics.

Syntax

```
METHOD MoveToPosition : BOOL
VAR_INPUT
    slogMessage      : STRING;
    targetPosition   : REFERENCE TO PositionXYC;
    velocity          : LREAL;
    acceleration      : LREAL;
    deceleration      : LREAL;
    jerk              : LREAL;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
slogMessage	STRING	Log info for the command
targetPosition	REFERENCE TO PositionXYC	Target position (coordinates)
velocity	LREAL	Velocity value
acceleration	LREAL	Acceleration rate
deceleration	LREAL	Deceleration rate
jerk	LREAL	Jerk limitation value

MoveZ

Lifts or lowers the mover over the stator surface.

Syntax

```
METHOD MoveZ : BOOL
VAR_INPUT
    slogMessage      : STRING;
    targetPosition   : LREAL;
    velocity          : LREAL;
    acceleration      : LREAL;
    deceleration      : LREAL;
    jerk              : LREAL;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
slogMessage	STRING	Log info for the command
targetPosition	LREAL	Target levitation height over the surface
velocity	LREAL	Velocity value
acceleration	LREAL	Acceleration rate
deceleration	LREAL	Deceleration rate
jerk	LREAL	Jerk limitation value

Reset

Resets any pending error on the mover. Applicable whenever the mover switches to the Error state.

Syntax

```
METHOD Reset : BOOL
VAR_INPUT
    slogMessage : STRING;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
slogMessage	STRING	Log info for the command

SetExternalSetpointValues

External setpoint values set by the external setpoint generator. This method must be called every PLC cycle while the external setpoint generator mode is enabled. The mover attempts to follow these setpoint values without consideration for Collision Avoidance monitoring.

Syntax

```
METHOD SetExternalSetpointValues : BOOL
VAR_INPUT
    position      : MoverVector;
    velocity      : MoverVector;
    acceleration  : MoverVector;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
position	MoverVector	Position setpoint 6D vector
velocity	MoverVector	Velocity setpoint 6D vector
acceleration	MoverVector	Acceleration setpoint 6D vector

UpdateCommandHistory

Appends a log info entry into the log history.

Syntax

```
METHOD UpdateCommandHistory
VAR_IN_OUT
    stCommand      : ST_MoverCommand;
END_VAR
```

In/Outputs

Name	Type	Description
stCommand	ST_MoverCommand	Command info to be logged

 Properties

Name	Type	Access	Description
P_ActPosition	REFERENCE TO MoverVector	RO	Returns the actual position 6D vector
P_ActVelocity	REFERENCE TO MoverVector	RO	Returns the actual velocity 6D vector
P_AdoptTrackOrientation	REFERENCE TO ST_CommandReturn	RO	Command feedback for AdoptTrackOrientation
P_AtRotationPoint	BOOL	RO	Returns if the mover is allowed to rotate (C rotation) at its current location
P_Busy	Bool	RO	Returns if a command (movement) is busy executing
P_ClosestQuarterAngle	LREAL	RO	Returns the closest 90° angle value (shortest way)
P_CommandMode	MC_PLANAR_MOVER_COMMAND_MODE	RO	Returns the mover's current operation mode
P_CRotationRestrictions	BOOL	RO	Returns if the mover is allowed to move in X-Y directions, based on its current C orientation
P_Disable	REFERENCE TO ST_CommandReturn	RO	Command feedback for Disable
P_DisableExternalSetpointGeneration	REFERENCE TO ST_CommandReturn	RO	Command feedback for DisableExternalSetpointGeneration
P_Enable	REFERENCE TO ST_CommandReturn	RO	Command feedback for Enable
P_EnableExternalSetpointGeneration	REFERENCE TO ST_CommandReturn	RO	Command feedback for EnableExternalSetpointGeneration
P_Halt	REFERENCE TO ST_CommandReturn	RO	Command feedback for Halt
P_HaltDynamics	ST_HaltDynamics	RW	Dynamic constraint values used by Halt command
P_Id	UINT	RW	Returns the mover ID (1 to n)
P_InGroup	BOOL	RO	Returns whether the mover is part of the Collision Avoidance group
P_IsQueuing	BOOL	RO	Returns if the mover is queueing-up during a move command
P_JoinTrack	REFERENCE TO ST_CommandReturn	RO	Command feedback for JoinTrack
P_LeaveTrack	REFERENCE TO ST_CommandReturn	RO	Command feedback for LeaveTrack
P_MoveA	REFERENCE TO ST_CommandReturn	RO	Command feedback for MoveA
P_MoveB	REFERENCE TO ST_CommandReturn	RO	Command feedback for MoveB
P_MoveC	REFERENCE TO ST_CommandReturn	RO	Command feedback for MoveC
P_MoveOnTrack	REFERENCE TO ST_CommandReturn	RO	Command feedback for MoveOnTrack

Name	Type	Access	Description
P_MoverOid	OTCID	RO	Returns the mover's NC object ID
P_MoveToPosition	REFERENCE TO ST_CommandReturn	RO	Command feedback for MoveToPosition
P_MoveZ	REFERENCE TO ST_CommandReturn	RO	Command feedback for MoveZ
P_Reset	REFERENCE TO ST_CommandReturn	RO	Command feedback for Reset
P_RotationAngleModulo	LREAL	RO	Returns the actual C angle within modulo range (°)
P_SetAcceleration	REFERENCE TO MoverVector	RO	Returns the current set acceleration rate
P_SetOnTrack	REFERENCE TO CDT_MCTOPLC_PLANAR_MOVER_TRACK	RO	Returns the current set values for OnTrack movement
P_SetPosition	REFERENCE TO MoverVector	RO	Returns the current set position value
P_SetVelocity	REFERENCE TO MoverVector	RO	Returns the current set velocity value
P_State	MC_PLANAR_STATE	RO	Returns the current mover state
P_TrackId	UDINT	RO	Returns the current track ID when mover is on a track

FB_XPlanarSystem

The object handles the system wide relationships and operations between movers, tracks, and the environment.

Do not call the main FB directly. Only use the available methods and properties.

Methods

Name	Description
AddEnvironmentToGroup	Adds an environment to a group
AddModuleToEnvironment	Adds a tile to an environment
AddMoversToGroup	Adds all movers to a group
AddMoverToGroup	Adds one mover to a group
AddTracksToGroup	Adds all tracks to a group
CheckValidMovers	Checks if movers are still valid in the system
Cycle	Cyclic operations of FB_XPlanarSystem
DisableGroup	Disables a group
DisableMovers	Disables all movers in a group
DisableTracks	Disables all tracks in a group
EnableGroup	Enables a group
EnableMovers	Enables all mover in a group
EnableTrack	Enables a specific track
EnableTracks	Enables all tracks in a group
HaltSystem	Halts all movers in a group
LeaveTrack	All movers leave their current track
RemoveEnvironmentFromGroup	Removes an environment from a group
RemoveMoverFromGroup	Removes a mover from a group
RemoveMoversFromGroup	Removes all movers from a group
RemoveTracksFromGroup	Removes all tracks from a group
ResetGroup	Resets a group
ResetMovers	Resets all errored movers
ResetTracks	Resets all errored tracks
SetMoverStandard	Adds a mover reference to the motion system
SetTrackStandard	Adds a track reference to the motion system

AddEnvironmentToGroup

Adds the stator surface boundaries definition, called Environment, to the Collision Avoidance group.

Syntax

```
METHOD AddEnvironmentToGroup : BOOL
```

Return value

BOOL

AddModuleToEnvironment

Adds a tile, referenced by its lower left corner coordinates X and Y, to the stator surface definition.

Syntax

```
METHOD AddModuleToEnvironment : BOOL
VAR_INPUT
    xPos    : LREAL;
    yPos    : LREAL;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
xPos	LREAL	X coordinate of the lower left corner
yPos	LREAL	Y coordinate of the lower left corner

AddMoversToGroup

Adds all enabled movers to the Collision Avoidance group.

Syntax

```
METHOD AddMoversToGroup : BOOL
```

Return value

BOOL

AddMoverToGroup

Adds the specified enabled mover to the Collision Avoidance group.

Syntax

```
METHOD AddMoverToGroup : BOOL
VAR_INPUT
    ipMover    : I_XPlanarMoverStandard;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
ipMover	I_XPlanarMoverStandard	Mover object to be added to the C.A. group

AddTracksToGroup

Adds all enabled tracks to the Collision Avoidance group.

Syntax

```
METHOD AddTracksToGroup : BOOL
```

Return value

BOOL

CheckValidMovers

Checks whether all movers are still present on the surface and returns how many actual movers have been found.

Syntax

```
METHOD CheckValidMovers : BOOL
```

Return value

BOOL

Cycle

Cyclic update operations. Handles communication with the process image. Method must be called every PLC cycle.

Syntax

```
METHOD Cycle
```

DisableGroup

Disables the Collision Avoidance group. Collision Avoidance monitoring gets stopped and collisions during movements are not prevented anymore.

Syntax

```
METHOD DisableGroup : BOOL
```

Return value

BOOL

DisableMovers

Disables all movers. Movers are not halted (freewheel stop).

Syntax

```
METHOD DisableMovers : BOOL
```

Return value

BOOL

DisableTracks

Disables all tracks.

Syntax

```
METHOD DisableTracks : BOOL
```

Return value

BOOL

EnableGroup

Enables the Collision Avoidance group. Depending on how many elements are included in the group, Collision Avoidance monitoring resumes.

Syntax

```
METHOD EnableGroup : BOOL
```

Return value

BOOL

EnableMovers

Enables all movers.

Syntax

```
METHOD EnableMovers : BOOL
```

Return value

BOOL

EnableTrack

Enables the specified track.

Syntax

```
METHOD EnableTrack : BOOL
VAR_INPUT
    ipTrackToAdd : I_TrackStandard;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
ipTrackToAdd	I_TrackStandard	Track object to enable

EnableTracks

Enables all tracks.

Syntax

```
METHOD EnableTracks : BOOL
```

Return value

BOOL

HaltSystem

Halts all movers present in the system.

Syntax

```
METHOD HaltSystem : BOOL
```

Return value

BOOL

LeaveTrack

Sends the LeaveTrack command to all movers in the system.

Syntax

```
METHOD LeaveTrack : BOOL
```

Return value

BOOL

RemoveEnvironmentFromGroup

Removes the stator surface boundaries from the Collision Avoidance group.

Syntax

```
METHOD RemoveEnvironmentFromGroup : BOOL
```

Return value

BOOL

RemoveMoverFromGroup

Removes the specified mover from the Collision Avoidance group.

Syntax

```
METHOD RemoveMoverFromGroup : BOOL  
VAR_INPUT  
    ipMover      : I_XPlanarMoverStandard;  
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
ipMover	I_XPlanarMover Standard	Mover object to be removed from C.A. group

RemoveMoversFromGroup

Removes all movers from the Collision Avoidance group.

Syntax

```
METHOD RemoveMoversFromGroup : BOOL
```

Return value

BOOL

RemoveTracksFromGroup

Removes all tracks from the Collision Avoidance group.

Syntax

```
METHOD RemoveTracksFromGroup : BOOL
```

Return value

BOOL

ResetGroup

Resets any pending error state on the Collision Avoidance group object. This command does not reset the errors on the elements included in the group.

Syntax

```
METHOD ResetGroup : BOOL
```

Return value

BOOL

ResetMovers

Resets any pending error state on all movers.

Syntax

```
METHOD ResetMovers : BOOL
```

Return value

BOOL

ResetTracks

Resets any pending error state on all tracks.

Syntax

```
METHOD ResetTracks : BOOL
```

Return value

BOOL

SetMoverStandard

Adds a mover object to the system. All declared mover objects must be included in the system with this method at startup. Mover index order is important and must be considered.

Syntax

```
METHOD SetMoverStandard : I_XPlanarMoverStandard
VAR_INPUT
    nnum          : UDINT;
    ipMoverStandard : I_XPlanarMoverStandard;
END_VAR
```

Return value

I_XPlanarMoverStandard (unused)

Inputs

Name	Type	Description
nnum	UDINT	Mover index (1 to n)
ipMoverStandard	I_XPlanarMoverStandard	Mover object to be included in the system

SetTrackStandard

Adds a track object to the system. All declared track objects must be included in the system with this method at startup. Track index order is important and must be considered.

Syntax

```
METHOD SetTrackStandard : Bool
VAR_INPUT
    nnum          : UDINT;
    ipTrackStandard : I_TrackStandard;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
nnum	UDINT	Track index (1 to n)
ipTrackStandard	I_TrackStandard	Track object to be included in the system

 Properties

Name	Type	Access	Description
P_AddEnvironmentToGroup	REFERENCE TO ST_CommandReturn	RO	Command feedback for AddEnvironment
P_AddModuleToEnvironment	REFERENCE TO ST_CommandReturn	RO	Command feedback for AddModuleToEnvironment
P_AddMoversToGroup	REFERENCE TO ST_CommandReturn	RO	Command feedback for AddMoversToGroup
P_AddMoverToGroup	REFERENCE TO ST_CommandReturn	RO	Command feedback for AddMoverToGroup
P_AddTracksToGroup	REFERENCE TO ST_CommandReturn	RO	Command feedback for AddTracksToGroup
P_CheckValidMovers	REFERENCE TO ST_MoverCheckInfo	RO	Command feedback for CheckValidMover
P_DisableGroup	REFERENCE TO ST_CommandReturn	RO	Command feedback for DisableGroup
P_DisableMovers	REFERENCE TO ST_CommandReturn	RO	Command feedback for DisableMovers
P_DisableTracks	REFERENCE TO ST_CommandReturn	RO	Command feedback for DisableTracks
P_EnableGroup	REFERENCE TO ST_CommandReturn	RO	Command feedback for EnableGroup
P_EnableMovers	REFERENCE TO ST_CommandReturn	RO	Command feedback for EnableMovers
P_EnableTrack	REFERENCE TO ST_CommandReturn	RO	Command feedback for EnableTrack
P_EnableTracks	REFERENCE TO ST_CommandReturn	RO	Command feedback for EnableTracks
P_HaltSystem	REFERENCE TO ST_CommandReturn	RO	Command feedback for HaltSystem
P_LeaveTrack	REFERENCE TO ST_CommandReturn	RO	Command feedback for LeaveTrack
P_RemoveEnvironmentFromGroup	REFERENCE TO ST_CommandReturn	RO	Command feedback for RemoveEnvironmentFromGroup
P_RemoveMoverFromGroup	REFERENCE TO ST_CommandReturn	RO	Command feedback for RemoveMoverFromGroup
P_RemoveMoversFromGroup	REFERENCE TO ST_CommandReturn	RO	Command feedback for RemoveMoversFromGroup
P_RemoveTracksFromGroup	REFERENCE TO ST_CommandReturn	RO	Command feedback for RemoveTracksFromGroup
P_ResetGroup	REFERENCE TO ST_CommandReturn	RO	Command feedback for ResetGroup
P_ResetMovers	REFERENCE TO ST_CommandReturn	RO	Command feedback for ResetMovers
P_ResetTracks	REFERENCE TO ST_CommandReturn	RO	Command feedback for ResetTracks

Name	Type	Access	Description
ipDriver	I_Environment	RO	Returns the XPlanar driver interface

FB_XPlanarVisuDiag

Handles the visualization data for the XPlanar application.

Do not call the main FB directly. Only use the available methods and properties.

Methods

Name	Description
Cycle	Cyclic update operation
SetStandardMover	Adds a mover object to be observed with the PLC HMI
SetStandardTrack	Adds a track object to be observed with the PLC HMI
UpdateMoverStatus	Updates the mover status info

Cycle

Cyclic update operations. This method must be called every PLC cycle to refresh the PLC HMI info data.

Syntax

```
METHOD SampleMethod : BOOL
```

Return value

BOOL

SetStandardMover

Adds a mover object to the visualization info data.

Syntax

```
METHOD SetStandardMover : BOOL
VAR_INPUT
    ipMover      : I_XPlanarMoverStandard;
    nNum         : UDINT;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
ipMover	I_XPlanarMoverStandard	Mover object to be observed
nNum	UDINT	Mover index (1 to n)

SetStandardTrack

Adds a track object to the visualization info data.

Syntax

```
METHOD SetStandardTrack : BOOL
VAR_INPUT
    ipTrack      : I_TrackStandard;
    nNum         : UDINT;
END_VAR
```

Return value

BOOL

Inputs

Name	Type	Description
ipTrack	I_TrackStandard	Track object to be observed
nNum	UDINT	Track index (1 to n)

UpdateMoverStatus

Updates the mover status info.

Syntax

```
METHOD UpdateMoverStatus : ST_MoverVisualisation
```

Return value

ST_MoverVisualisation